Modern Education Society's

Nowrosjee Wadia College (AUTONOMOUS)

Affiliated to the **Savitribai Phule Pune University**
(Formerly University of Pune)

As per National Education Policy 2020 NEP 1.0

To be implemented from Academic Year 2025-2026

T. Y. B. Sc. (Computer Science)
SEMESTER VI

CSMJ365 Laboratory Course on
Advanced Java programming (CSMJ363)
and
Data Analytics (CSMJ362)

Student Name:	
Roll No:	
Division:	
Year:	

Internal Examiner _____ External Examiner _____

Prepared By,

Mr. Govind A. Hadke

Asst. Prof. Deparment Of Computer Science,

Nowrosjee Wadia College, Pune

Co-Ordinator

Mrs. Vandana Bhatt

Department Of Computer Science,

Nowrosjee Wadia College, Pune

Department Head,

Dr. Poonam Ponde

Department Of Computer Science

Nowrosjee Wadia College, Pune

Assignment completion Sheet

Advance Java Programming			
Sr. No	**Assignment Name**	**Marks (Out of 5)**	**Signature**
1	Collections		
2	Multithreading		
3	JDBC		
4	Servlet JSP		
Data Analytics			
1	Linear and Logistic Regression		
2	Frequent itemset and Association rule mining		
3	Text and Social Media Analytics		
Total out of 35 (Assignments on Advanced Java (20) + Assignments on Data Analytics (15))			
Convert into 10 Marks			
Viva (5 Marks)			
Total out of 15			

This is to certify that Ms. / Mr. _____

Roll Number_____has successfully completed the course workfor CSMJ365 and has scored_____marks out of 15.

Instructor **Head**

Internal Examiner **External Examiner**

Introduction

About the workbook

This workbook is intended to be used by T. Y. B. Sc (CS) students for the Laboratory Course CSMJ365 – based on Advanced JAVA programming (CSMJ363) and Data Analytics (CSMJ362)

The objectives of this book are

- Defining clearly the scope of the course
- Continuous assessment of the Students.
- Bring variation and variety in experiments carried out by different students in a batch
- Providing ready reference for students while working in the lab
- Catering to the need of slow paced as well as fast paced learners

How to use this workbook

The Advanced Java, practical syllabus is divided into five assignments. Each assignment has problems divided into three sets A, B and C.

- Set A is used for implementing the basic algorithms or implementing data structure along with its basic operations. **Set A is mandatory.**
- Set B is used to demonstrate small variations on the implementations carried out in set A to improve its applicability. Depending on the time availability the students should be encouraged to complete set B.
- Set C prepares the students for the viva in the subject. Students should spend additional time either at home or in the Lab and solve these problems so that they get a deeper understanding of the subject.

Instructions to the students

- Please read the following instructions carefully and follow them.
- Students are expected to carry workbook during every practical.
- Students should prepare oneself beforehand for the Assignment by reading the relevant material.
- Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.

- Students will be assessed for each exercise on a scale from 0 to 5
  - Not done 0
  - Incomplete 1
  - Late Complete 2
  - Needs improvement 3
  - Complete 4
  - Well Done 5

Instruction to the Practical In-Charge

- Explain the assignment and related concepts in around ten minutes using white board if required or by demonstrating the software.
- Choose appropriate problems to be solved by students. Set A is mandatory. Choose problems from set B depending on time availability. Discuss set C with students and encourage them to solve the problems by spending additional time in lab or at home.
- Make sure that students follow the instruction as given above.
- You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- The value should also be entered on assignment completion page of the respective Lab course.

Instructions to the Lab Administrator and Exam Guidelines

- You have to ensure appropriate hardware and software is made available to each student.
- Do not provide Internet facility in Computer Lab while examination
- Do not provide pen drive facility in Computer Lab while examination.

The operating system and software requirements are as given below:

- Operating system: Linux
- Editor: Any linux based editor like vi, gedit , eclipse etc.
- Compiler: javac (**Note : JAVA 8 and above version**)

Section I

CSMJ363
Advanced Java Programming

Assignment No 1 : Collection

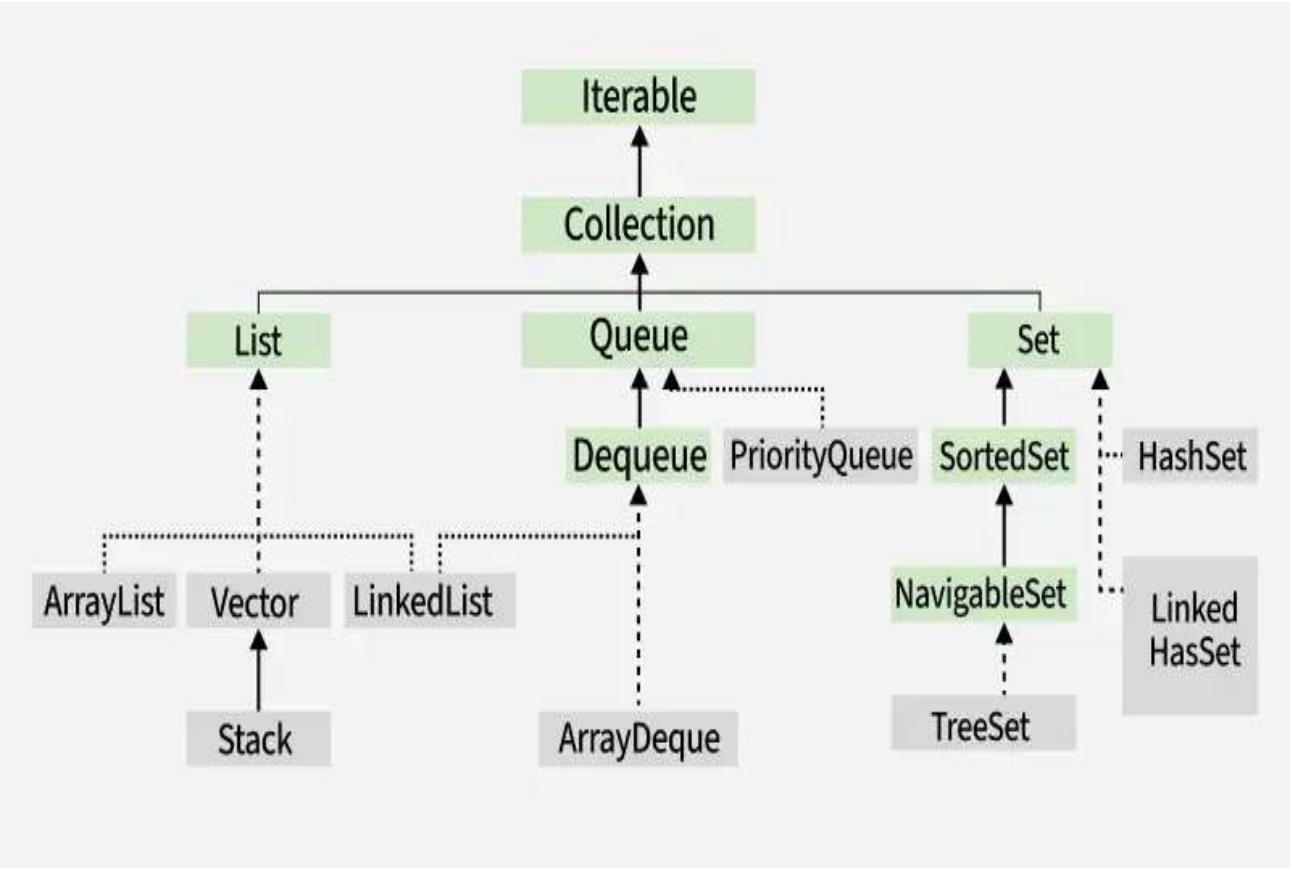
The **Java Collection framework** provides implementations for common data structures and algorithms. It includes interfaces and classes that simplify working with data.

Collection is representation of group of elements as single entity.

Why We Use Collection Interface Instead of Arrays?

Arrays	Collection
Fixed size	Dynamic size
Only store similar type	Can store objects
No inbuilt methods	Rich set of methods
Hard to insert/delete	Easy to insert/delete
Fast	Slight overhead

Collection Hierarchy



Collection

The Collection interface is the root of the Java Collections Framework, defined in the java.util package. It represents a group of individual objects as a single unit and provides basic operations for working with them.

- **Dynamic in Nature:** Collections can automatically grow or shrink in size, unlike arrays that have a fixed length.
- **Stores Homogeneous and Heterogeneous Objects:** Can hold same-type or different-type elements based on implementation.
- **Easy to Use:** Provides convenient methods such as add(), remove(), and clear() to manage elements effortlessly.
- **Efficient Traversal:** Allows easy access and processing of elements using loops or iterators.

Methods of Collection Interface:

Method	Description
add(E e)	Adds an element to the collection (optional).
addAll(Collection<? extends E> c)	Adds all elements from another collection.
clear()	Removes all elements.
contains(Object o)	Checks if an element exists.
containsAll(Collection<?> c)	Checks if all elements exist.
remove(Object o)	Removes a single instance of the specified element.
removeAll(Collection<?> c)	Removes all elements present in another collection.
retainAll(Collection<?> c)	Retains only elements present in another collection.
removeIf(Predicate<? super E> filter)	Removes elements satisfying a predicate.
size()	Returns the number of elements.
isEmpty()	Checks if the collection is empty.
iterator()	Returns an iterator over the elements.

Method	Description
stream()	Returns a sequential stream.
parallelStream()	Returns a parallel stream.
toArray()	Converts the collection to an array.
equals(Object o)	Compares this collection with another object.
hashCode()	Returns hash code of the collection.

Example: Collection using ArrayList

```
import java.util.Collection;
import java.util.ArrayList;
import java.util.Iterator;

public class CollectionExample {
   public static void main(String[] args) {
      Collection<String> names = new ArrayList<>();

      names.add("Sahadev");
      names.add("Amit");
      names.add("Riya");

      System.out.println("Size: " + names.size());

      Iterator<String> itr = names.iterator();
      while (itr.hasNext()) {
         System.out.println(itr.next());
      }

      names.remove("Amit");
      System.out.println("After removal: " + names);
   }
}
```

List:

The List interface in Java extends the Collection interface and is part of the java.util package. It is used to store ordered collections where duplicates are allowed and elements can be accessed by their index.

- Maintains insertion order
- Allows duplicate elements
- Supports null elements (implementation dependent)
- Supports bidirectional traversal using ListIterator

```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Creating a List of Strings using ArrayList
        List<String> li = new ArrayList<>();

        // Adding elements in List
        li.add("Java");
        li.add("Python");
        li.add("DSA");
        li.add("C++");

        System.out.println("Elements of List are:");

        // Iterating through the list
        for (String s : li) {
            System.out.println(s);
        }

}
}
```

Methods of the List Interface

Methods	Description
add(int index, element)	This method is used with Java List Interface to add an element at a particular index in the list. When a single parameter is passed, it simply adds the element at the end of the list.
addAll(int index, Collection collection)	This method is used with List Interface in Java to add all the elements in the given collection to the list. When a single parameter is passed, it adds all the elements of the given collection at the end of the list.
size()	This method is used with Java List Interface to return the size of the list.

Methods	Description
clear()	This method is used to remove all the elements in the list. However, the reference of the list created is still stored.
remove(int index)	This method removes an element from the specified index. It shifts subsequent elements(if any) to left and decreases their indexes by 1.
remove(element)	This method is used with Java List Interface to remove the first occurrence of the given element in the list.
get(int index)	This method returns elements at the specified index.
set(int index, element)	This method replaces elements at a given index with the new element. This function returns the element which was just replaced by a new element.
indexOf(element)	This method returns the first occurrence of the given element or -1 if the element is not present in the list.
lastIndexOf(element)	This method returns the last occurrence of the given element or -1 if the element is not present in the list.
equals(element)	This method is used with Java List Interface to compare the equality of the given element with the elements of the list.
hashCode()	This method is used with List Interface in Java to return the hashcode value of the given list.
isEmpty()	This method is used with Java List Interface to check if the list is empty or not. It returns true if the list is empty, else false.
contains(element)	This method is used with List Interface in Java to check if the list contains the given element or not. It returns true if the list contains the element.
containsAll(Collection collection)	This method is used with Java List Interface to check if the list contains all the collection of elements.
sort(Comparator comp)	This method is used with List Interface in Java to sort the elements of the list on the basis of the given comparator.

Example: List using ArrayList

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListExample {
   public static void main(String[] args) {

      List<String> fruits = new ArrayList<>();

      // Adding elements
      fruits.add("Apple");
      fruits.add("Banana");
      fruits.add("Mango");

      // Display List
      System.out.println("ArrayList: " + fruits);

      // Access element
      System.out.println("First Element: " + fruits.get(0));

      // Remove element
      fruits.remove("Banana");
      System.out.println("After Removal: " + fruits);
   }
}
```

ArrayList

- Description: Dynamic array implementation in Java.
- Key Features:
  - Allows random access via an index.
  - Resizable; grows dynamically when needed.
  - Suitable for fast read operations but slower for insertion/deletion compared to LinkedList.
- Common Operations:
  - add(element): Adds an element to the list.
  - remove(index): Removes the element at the specified index.
  - get(index): Retrieves the element at the specified index.
  - size(): Returns the size of the list.

Example: Operations on arraylist

```
import java.util.ArrayList;

public class ArrayListMethodsExample {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        // 1. add() → Add elements
        names.add("");
        names.add("Amit");
        names.add("Riya");
        names.add("Neha");
        System.out.println("After add(): " + names);

        // 2. add(index, element) → Insert at position
        names.add(2, "Sagar");
        System.out.println("After add(index, element): " + names);

        // 3. get() → Fetch element
        System.out.println("Element at index 1: " + names.get(1));

        // 4. set() → Update element
        names.set(1, "Amit Kumar");
        System.out.println("After set(): " + names);

        // 5. remove(index) → Remove by index
        names.remove(3);
        System.out.println("After remove(index): " + names);

        // 6. remove(Object) → Remove by value
        names.remove("Sagar");
        System.out.println("After remove(object): " + names);

        // 7. contains() → Check existence
        System.out.println("Contains Neha? " + names.contains("Neha"));

        // 8. size() → Find number of elements
        System.out.println("Size of ArrayList: " + names.size());

        // 9. isEmpty() → Check empty or not
        System.out.println("Is empty? " + names.isEmpty());

        // 10. indexOf() → Find index of element
        System.out.println("Index of Neha: " + names.indexOf("Neha"));

        // 11. toArray() → Convert to array
```

```
        Object arr[] = names.toArray();
        System.out.print("Converted to Array: ");
        for(Object o : arr) {
           System.out.print(o + " ");
        }
        System.out.println();

        // 12. clear() → Remove all elements
        names.clear();
        System.out.println("After clear(): " + names);
        System.out.println("Is empty after clear? " + names.isEmpty());
    }
}
```

LinkedList

- Description: Doubly-linked list implementation.
- Key Features:
    - Efficient for insertion and deletion operations.
    - Maintains the order of elements.
    - Can be used as a List, Deque, or Queue.

Method	Description
add()	Adds element at the end
add(index, element)	Inserts at specific position
addFirst()	Adds element at beginning
addLast()	Adds element at end
get()	Get element by index
getFirst()	Get first element
getLast()	Get last element
set()	Replace element
remove()	Removes first occurrence
remove(index)	Removes by index
removeFirst()	Removes first element
removeLast()	Removes last element
contains()	Check if element exists
size()	Count elements
isEmpty()	Check empty or not
clear()	Remove all
indexOf()	Get position
toArray()	Convert to array

ArrayList V/S LinkedList:

Feature	ArrayList	LinkedList
Data Structure	Dynamic array (continuous memory)	Doubly linked list (nodes with pointers)
Access / Retrieval (get(index))	Very fast	Slow— must traverse
Insertion / Deletion in Middle	Slow — shifting required	Fast — simply change links
Insertion at End	Fast (unless array grows)	Fast
Memory Usage	Uses less memory (stores only data)	Uses more memory (stores data + 2 pointers)
Search (contains())	Faster	Slower
Best for	Frequent read operations (search & access by index)	Frequent insert & remove operations
Random Access	Supported	Not supported
Iterating	Fast	Slight overhead
Use Case Example	Student list, product list, reports page	Queue, playlist, undo operations
Internal Growth	Grows 50% of current size when full	No resizing needed
Null Elements	Allowed	Allowed
Implements	List	List & Deque

Example : Operations on LinkedList

```
import java.util.LinkedList;

public class LinkedListMethodsExample {

    public static void main(String[] args) {

        LinkedList<String> friends = new LinkedList<>();

        // 1. add() - add elements
        friends.add("Ravi");
        friends.add("Amit");
        friends.add("Neha");
        System.out.println("After add(): " + friends);

        // 2. add(index, element)
        friends.add(1, "Sagar");
        System.out.println("After add(index, element): " + friends);

        // 3. addFirst() and addLast()
        friends.addFirst("FirstFriend");
```

```
        friends.addLast("LastFriend");
        System.out.println("After addFirst and addLast: " + friends);

        // 4. get()
        System.out.println("Element at index 2: " + friends.get(2));

        // 5. getFirst() and getLast()
        System.out.println("First Element: " + friends.getFirst());
        System.out.println("Last Element: " + friends.getLast());

        // 6. set() - update value
        friends.set(2, "UpdatedFriend");
        System.out.println("After set(): " + friends);

        // 7. remove() and remove(index)
        friends.remove("Amit");
        friends.remove(3);
        System.out.println("After remove(): " + friends);

        // 8. removeFirst() and removeLast()
        friends.removeFirst();
        friends.removeLast();
        System.out.println("After removeFirst and removeLast: " + friends);

        // 9. contains()
        System.out.println("Contains Neha? " + friends.contains("Neha"));

        // 10. size()
        System.out.println("Size: " + friends.size());

        // 11. isEmpty()
        System.out.println("Is empty? " + friends.isEmpty());

        // 12. indexOf()
        System.out.println("Index of Neha: " + friends.indexOf("Neha"));

        // 13. toArray()
        Object arr[] = friends.toArray();
        System.out.print("Array: ");
        for(Object o : arr){
           System.out.print(o + " ");
        }
        System.out.println();

        // 14. clear() - remove all
        friends.clear();
        System.out.println("After clear(): " + friends);
        System.out.println("Is Empty after clear? " + friends.isEmpty());
    }
}
```

Set:

The Set interface represents a collection of unique elements.
It does not allow duplicate values and is used when we want distinct data.

Feature	Description
No duplicates	Every element is unique
Order not guaranteed	(Depends on implementation)
Allows null values	Depends on Set type
Cannot access by index	No get(index) method
Fast lookup	Especially HashSet

Methods:

Method	Description
add(E e)	Add element
addAll(Collection c)	Add a collection
remove(Object o)	Remove element
contains(Object o)	Check exists
size()	Count
isEmpty()	Check empty
clear()	Remove all
iterator()	Traverse elements

HashSet:

HashSet is a class in Java that implements the Set interface and stores elements in an unordered collection.

➞ It does not allow duplicate elements
➞ It is based on a hash table
➞ It is faster for search, add, and remove operations

Feature	Description
No duplicates allowed	Automatically ignores duplicates
No guaranteed order	Output order is random
Null allowed	One null element
Fast performance	Uses hashing
No index access	Cannot use get(index)
Allows heterogeneous elements	But not recommended

Methods:

Method	Description
add(E e)	Add element
remove(Object o)	Remove element
contains(Object o)	Check presence
size()	Count elements
isEmpty()	Check empty
clear()	Removes all
iterator()	To traverse

Example

```
import java.util.HashSet;

public class HashSetExample {
   public static void main(String[] args) {

       HashSet<String> languages = new HashSet<>();

       // Adding elements
       languages.add("Java");
       languages.add("C");
       languages.add("Python");
       languages.add("Java");  // Duplicate - ignored
       languages.add(null);

       System.out.println("Languages Set: " + languages);

       // Check presence
       System.out.println("Contains Python? " +
languages.contains("Python"));

       // Remove element
       languages.remove("C");
       System.out.println("After Removal: " + languages);

       System.out.println("Size: " + languages.size());
   }
}
```

Advantages of HashSet

Fast operations (add, remove, search)
No duplicates → ensures uniqueness

Good for membership testing (contains())
Efficient for large data set

Limitations of HashSet

No ordering of elements
No index-based access
Performance depends on good hashCode() implementation

SortedSet

SortedSet is an interface in Java that extends the Set interface and stores unique elements in sorted (ascending) order**.**

It automatically arranges elements in natural order (numbers ascending, strings alphabetically).

It does not allow duplicate values**.**

Methods

Method	Description
first()	Returns smallest element
last()	Returns largest element
headSet(E e)	Returns elements < e
tailSet(E e)	Returns elements ≥ e
subSet(E from, E to)	Range elements
comparator()	Shows comparison logic
add(), remove(), contains()	Inherited from Set
size(), clear(), isEmpty()	Inherited from Set

Navigable Set:

NavigableSet is a sub interface of SortedSet introduced in Java 6.
It provides navigation methods to search elements based on closest match (lower, higher, floor, ceiling) in addition to sorted operations.
It allows retrieving, removing, checking, and iterating elements in both forward and reverse order**.**
Methods:

Method	Description
E lower(E e)	Returns the element less than given element.
E floor(E e)	Returns the element <= given element.
E ceiling(E e)	Returns the element >= given element.

Method	Description
E higher(E e)	Returns the element greater than given element.
E pollFirst()	Returns and removes first (lowest) element.
E pollLast()	Returns and removes last (highest) element.
NavigableSet<E> descendingSet()	Returns set elements in reverse order.
Iterator<E> descendingIterator()	Returns reverse-order iterator.
NavigableSet<E> subSet(E from, boolean incl1, E to, boolean incl2)	Returns a portion of set between range with include/exclude options.
headSet(E toElement, boolean inclusive)	Portion from start to given element.
tailSet(E fromElement, boolean inclusive)	Portion from given element to end.

Example : Navigable using treeset

```
import java.util.*;
public class NavigableSetExample {
   public static void main(String[] args) {
      NavigableSet<Integer> ns = new TreeSet<>();
      ns.add(10);
      ns.add(20);
      ns.add(30);
      ns.add(40);
      System.out.println("Lower(30) : " + ns.lower(30));     // Output: 20
      System.out.println("Floor(30) : " + ns.floor(30));     // Output: 30
      System.out.println("Ceiling(25): " + ns.ceiling(25));  // Output: 30
      System.out.println("Higher(30): " + ns.higher(30));    // Output: 40

      System.out.println("Descending Set: " + ns.descendingSet()); // [40, 30, 20, 10]
   }
}
```

TreeSet

TreeSet is a sorted, unique collection in Java that implements the NavigableSet and SortedSet interface.

- Description: Implements a sorted set based on a Red-Black Tree.
- Key Features:
  - Automatically sorts elements in ascending order.
  - No duplicate elements allowed.
  - Does not allow null elements.
  - Stores unique elements (no duplicates)

Feature	Description
Orders elements	Maintains sorted order
No duplicates	Accepts only unique elements
No null allowed	Throws NullPointerException
Slower than HashSet	Because of sorting
Implements NavigableSet	Extra navigation methods
Backed by TreeMap	Uses Red-Black Tree

Methods:

Method	Description
add(E e)	Add element
remove(Object o)	Remove element
contains(Object o)	Check presence
size()	Count elements
clear()	Remove all
first()	First (smallest) element
last()	Last (largest) element
higher(E e)	Just greater element
lower(E e)	Just smaller element
ceiling(E e)	≥ given element
floor(E e)	≤ given element
pollFirst()	Remove and return first
pollLast()	Remove and return last

Example:

```
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {

        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(10); // Duplicate ignored

        System.out.println("TreeSet: " + numbers);
```

```
        System.out.println("First: " + numbers.first());
        System.out.println("Last: " + numbers.last());
        System.out.println("Higher(20): " + numbers.higher(20));
        System.out.println("Lower(30): " + numbers.lower(30));

        numbers.remove(20);
        System.out.println("After removal: " + numbers);
    }
}
```

Advantages:

Elements always sorted
Provides navigation methods (higher, lower, etc.)
Useful for range searching and ordered collection

Disadvantage:

Slower than HashSet
No null allowed
Requires elements to be Comparable or must use Comparator

HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	No order (Random)	Maintains insertion order	Sorted (Ascending by default)
Duplicates	Not allowed	Not allowed	Not allowed
Null Value	Allows 1 null	Allows 1 null	✗ Does NOT allow
Performance (Add/Search/Delete)	Fastest (O(1))	Slightly slower than HashSet	Slowest (O(log n))
Underlying Data Structure	Hash Table	Hash Table + Linked List	Red-Black Tree
Index-based Access	Not supported	Not supported	Not supported
Sorting	✗ No	✗ No	✓ Yes (Natural or Custom)
Memory Usage	Low	Medium	High
Best Use Case	Unique data, no order needed	Maintain order of insertion	Unique + Sorted data
Example Use	IDs, email list	Recent history, cache	Dictionary, leaderboard

Queue:

The **Queue** interface in Java is a part of the **Java Collection Framework** and follows the **FIFO (First In First Out)** principle. Elements are inserted at the rear (tail) and removed from the front (head).

Queue is mainly used when elements need to be processed in the order they arrive, such as task scheduling, printer queue, CPU scheduling,

Key Characteristics of Queue

- Follows **FIFO** order (except PriorityQueue)
- Allows **duplicate elements**
- Generally does **not allow null** (PriorityQueue allows one null in older versions)
- Used for **order-based processing**

add(e) Inserts element, throws exception if fails

offer(e) Inserts element, returns false if fails

Iterator

An **Iterator** in Java is an interface that provides methods to iterate over a collection (like List, Set, etc.) in a sequential manner. It helps in accessing elements in a collection without exposing the underlying structure.

1. Iterator Interface

The **Iterator** interface is part of the java.util package and provides methods for iterating over a collection.

Key Methods of Iterator Interface:

1. hasNext()
    - Purpose: Returns true if there are more elements in the collection to iterate over, false otherwise.
    - Syntax: boolean hasNext();
2. next()
    - Purpose: Returns the next element in the collection and advances the iterator.
    - Syntax: E next();
    - Exception: Throws NoSuchElementException if no more elements are present.
3. remove()
    - Purpose: Removes the last element returned by the iterator from the underlying collection.
    - Syntax: void remove();
    - Exception: Throws IllegalStateException if next() has not been called yet or if remove() was already called after the last call to next().

2. Types of Iterators

1. Iterator for Collection (List, Set, etc.):
    - These iterators can iterate through elements of List, Set, Queue, etc.
    - Example:

```
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
   System.out.println(iterator.next());
}
```

2. ListIterator (for List collections only):
    - A ListIterator is a special type of iterator that allows traversing a list in both directions (forward and backward).
    - Additional Methods in ListIterator:
        - hasPrevious(): Checks if there are elements before the current position.
        - previous(): Returns the previous element and moves the cursor backward.
        - add(E e): Adds an element to the list at the current position.
        - set(E e): Replaces the last element returned by next() or previous() with the specified element.

Example: Using Iterator

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorExample {
   public static void main(String[] args) {
      // Create an ArrayList of Strings
      ArrayList<String> list = new ArrayList<>();
      list.add("Apple");
      list.add("Banana");
      list.add("Cherry");

      // Get an Iterator for the list
      Iterator<String> iterator = list.iterator();

      // Iterate through the list
      while (iterator.hasNext()) {
         System.out.println(iterator.next());
      }

      // Modify list using remove()
      iterator = list.iterator(); // Reset iterator
      while (iterator.hasNext()) {
         String item = iterator.next();
         if (item.equals("Banana")) {
```

```
            iterator.remove(); // Remove "Banana" from list
        }
    }

    // Print updated list
    System.out.println("Updated List: " + list);
  }
}
```

Example : Iterating over a List of Integers using for-each()

```
import java.util.ArrayList;
import java.util.List;

public class ForEachExample {
  public static void main(String[] args) {
    // Create a list of integers
    List<Integer> numbers = new ArrayList<>();

    // Add elements to the list
    numbers.add(1);
    numbers.add(2);
    numbers.add(3);
    numbers.add(4);
    numbers.add(5);

    // Iterate through the list using the for-each loop
    System.out.println("Iterating over the list using for-each loop:");
    for (Integer number : numbers) {
      System.out.println(number);
    }
  }
}
```

Collections Class

The **Collections** class is part of java.util package.It **consists of static utility methods** for operating on **Collection objects** like List, Set, or Map.

Purpose: To perform **common algorithms** (sorting, reversing, shuffling, searching, etc.) without manually writing code.

Key Features

- Provides methods for algorithmic operations on collections.
- Works with List, Set, and other Collection types.
- Contains methods for:
    - Sorting
    - Searching
    - Shuffling
    - Reversing
    - Rotating
    - Swapping
    - Filling
    - Finding max/min
    - Frequency count

Methods

Method	Description	Example
sort(List<T> list)	Sorts list in ascending order	Collections.sort(list);
sort(List<T> list, Comparator c)	Sorts list using custom comparator	Collections.sort(list, new MyComparator());
reverse(List<?> list)	Reverses the order of elements	Collections.reverse(list);
shuffle(List<?> list)	Randomly permutes elements	Collections.shuffle(list);
swap(List<?> list, int i, int j)	Swaps elements at indices i and j	Collections.swap(list, 0, 2);
max(Collection<? extends T> c)	Returns maximum element	Collections.max(list);
min(Collection<? extends T> c)	Returns minimum element	Collections.min(list);
frequency(Collection<?> c, Object o)	Counts occurrences of object	Collections.frequency(list, "Java");
fill(List<? super T> list, T obj)	Fills entire list with object	Collections.fill(list, "Default");
binarySearch(List<? extends Comparable<? super T>> list, T key)	Searches for key (list must be sorted)	Collections.binarySearch(list, "Python");
reverseOrder()	Returns comparator for reverse order	Collections.sort(list, Collections.reverseOrder());

Example : Operations using Collections class methods

```
import java.util.ArrayList;
import java.util.Collections;
```

```
public class CollectionsExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<String>();
        list.add("Java");
        list.add("Python");
        list.add("C++");
        list.add("Ruby");

        System.out.println("Original List: " + list);

        Collections.sort(list);
        System.out.println("Sorted List: " + list);

        Collections.reverse(list);
        System.out.println("Reversed List: " + list);

        Collections.shuffle(list);
        System.out.println("Shuffled List: " + list);

        Collections.swap(list, 0, 2);
        System.out.println("After Swap: " + list);

        System.out.println("Maximum Element: " + Collections.max(list));
        System.out.println("Minimum Element: " + Collections.min(list));

        System.out.println("Frequency of 'Java': " + Collections.frequency(list,
"Java"));

        Collections.fill(list, "Default");
        System.out.println("After Fill: " + list);
    }
}
```

Java Comparable and Comparator

Java, sorting of objects is achieved using **Comparable** or **Comparator** interfaces.

- Comparable – Used when a class has a natural ordering.
- Comparator – Used when you want custom ordering (external sorting logic).

Comparable Interface

- Part of `java.lang` package.
- Implemented by the class whose objects need to be sorted.
- Defines a single method: int compareTo(T obj)

Example: Sorting Employees by Name using Comparable

```
class Employee implements Comparable<Employee> {
    String name;
    int id;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int compareTo(Employee e) {
        return this.name.compareTo(e.name); // sort by name alphabetically
    }

    public String toString() {
        return id + " - " + name;
    }
}

import java.util.TreeSet;

public class ComparableExample {
    public static void main(String[] args) {
        TreeSet<Employee> employees = new TreeSet<Employee>();
        employees.add(new Employee(101, "Ajay"));
        employees.add(new Employee(102, "Reena"));
        employees.add(new Employee(103, "John"));

        for (Employee e : employees) {
            System.out.println(e);
        }
    }
}
```

Comparator Interface

Part of java.util package.
Used for custom sorting **or** multiple sorting sequences**.**

Example: Sorting Employees by ID using Comparator

```
import java.util.TreeSet;
import java.util.Comparator;

class Employee {
    String name;
    int id;
```

```
    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return id + " - " + name;
    }
}

class SortById implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        return e1.id - e2.id; // sort by id ascending
    }
}

public class ComparatorExample {
    public static void main(String[] args) {
        TreeSet<Employee> employees = new TreeSet<Employee>(new SortById());
        employees.add(new Employee(101, "Ajay"));
        employees.add(new Employee(102, "Reena"));
        employees.add(new Employee(103, "John"));

        for (Employee e : employees) {
            System.out.println(e);
        }
    }
}
```

What is Map?

- A Map is part of the Java Collection Framework.
- It is used to store key-value pairs.
- Each key is unique, but values can be duplicate.
- Think of it like a dictionary: you search for a value using a unique key.

Key Characteristics:

Feature	Description
Key	Must be **unique**
Value	Can be **duplicate**
Null Key	Allowed only once in `HashMap`/`LinkedHashMap`, not in `TreeMap`
Null Values	Allowed
Ordering	Depends on the Map type (unordered/sorted/insertion order)

Commonly Used Methods in Map

Method	Description
put(K key, V value)	Add or update a key-value pair
get(Object key)	Get the value for the given key
remove(Object key)	Remove the entry with the given key
containsKey(Object key)	Checks if a key exists
containsValue(Object value)	Checks if a value exists
keySet()	Returns a Set of all keys
values()	Returns a Collection of all values
entrySet()	Returns a Set of all key-value pairs

Map Interface Types in Java

Map Type	Order Maintained	Null Keys	Performance
HashMap	❌ No order	✅ 1 allowed	Fast
LinkedHashMap	✅ Insertion order	✅ 1 allowed	Slightly slower
TreeMap	✅ Sorted by keys	❌ Not allowed	Slower due to sorting
Hashtable	❌ No order	❌ Not allowed	Synchronized (Thread-safe)

Example 1: HashMap

```
import java.util.*;

public class HashMapDemo {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();

        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Mango");

        System.out.println("Value at key 2: " + map.get(2));  // Banana

        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }
    }
}
```

Example 2: TreeMap – Sorted by Keys

```
import java.util.*;

public class TreeMapDemo {
    public static void main(String[] args) {
        Map<String, Integer> map = new TreeMap<>();

        map.put("Banana", 30);
        map.put("Apple", 50);
        map.put("Mango", 20);

        System.out.println("Sorted map:");
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }
    }
}
```

Real-Life Use Cases of Map

Scenario	Description
Student Records	Roll No → Student Name
Login System	Username → Password
Product Info	Product ID → Product Name
Phonebook	Name → Phone Number

Key Points to Remember:

- Map is not a child interface of Collection, but part of the Java Collection Framework.
- You can't use add() like in List/Set. Instead, use put(key, value).
- Use Map.Entry<K,V> for iterating key-value pairs.
- TreeMap requires keys to be Comparable (or use a custom Comparator).

Example : TreeMap with User-Defined Class as Key

```
import java.util.*;

// User-defined class
class Student implements Comparable<Student> {
    int rollNo;
    String name;

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
```

```
    // Required for sorting in TreeMap
    @Override
    public int compareTo(Student s) {
        return Integer.compare(this.rollNo, s.rollNo); // sort by rollNo
    }

    @Override
    public String toString() {
        return "Roll: " + rollNo + ", Name: " + name;
    }
}

public class TreeMapWithCustomObject {
    public static void main(String[] args) {
        TreeMap<Student, String> map = new TreeMap<>();

        map.put(new Student(3, "Amit"), "Pune");
        map.put(new Student(1, "Rahul"), "Mumbai");
        map.put(new Student(4, "Priya"), "Nashik");
        map.put(new Student(2, "Sana"), "Delhi");

        for (Map.Entry<Student, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => City: " + entry.getValue());
        }
    }
}
```

Exercise :

SET A

1) Write a Java program to store 5 Strings in an ArrayList and display the list using iterator and foreach and perform the following operations: insert a string at a specific index, remove a given string, and check if a specific string exists in the list.

2) Write a Java Program to Create a HashSet and TressSet of integers display it by using foreach and iterator.

3) Write a Java program to create a Map<String, Integer> with some key-value pairs. Iterate and display the entries using both an Iterator and a for-each loop

4) Write a Java program to demonstrate basic operations on Stack. Perform push, pop on the Stack .

5) Write a Java program to create a List of integers, remove duplicates using Set, and display unique values.

6) Write a java program to accept numbers form user and sort them (duplicate can allowed and sort it) user appropriate collection classes

SET B

1) Write a Java Program that accept n integer values store in List and do the following operations.
   i) Display all elements of the list using an Iterator.
   ii) Count how many elements are Less than or equal to 20 & Greater than 20.
   iii) Remove all elements **greater than 20** while iterating..
   iv) Find and display: a. Maximum value b. Minimum value (after removal)
   v) Display the final updated list.

2) Write a Java program to store Employee objects in a TreeSet and automatically sort them alphabetically by name. Display the sorted list of employees. Use Comparable or Comparator for custom sorting.

3) Create a Map to store employee IDs and their names. Display all key-value pairs using Map.Entry and Sort them according to employee IDs.

4) Write a menu-driven Java program to store integer elements in an ArrayList. The program should use methods of the Collections class to perform operations such as sorting the list, reversing the list, swapping two elements, finding the maximum and minimum values, searching for an element, and displaying the frequency of a given element.

SET C

1) Write a Java program to accept a string, store the frequency of each character in a TreeMap, and display the characters in sorted order along with their counts.

2) Write a Java program to manage Student records using TreeSet sorted by name (using Comparator). Implement a menu-driven program to add, display, search by roll number, and delete by roll number.

3) Write a Java program to manage 10 integers using TreeSet<Integer>**.** The program should store numbers in sorted order, display them**,** find the smallest and largest**,** remove a number specified by the user**,** and demonstrate NavigableSet methods**:** lower(), higher(), ceiling(), floor(), pollFirst(), and pollLast().

Name and Signature of the Instructor: _____

Date of Evaluation: _____

Assignment Evaluation:

0. Not Done: ☐ 1. Incomplete : ☐ 2. Late Complete : ☐

3. Needs Improvement : ☐ 4. Complete : ☐ 5. Well Done : ☐

Assignment No 2 : Multithrading

What is Multithreading?

Multithreading allows multiple threads to run concurrently, making efficient use of CPU. Each thread is an independent path of execution.

Multithreading is a feature in Java that allows concurrent execution of two or more threads for maximum CPU utilization.

- A thread is the smallest unit of a process.
- Multiple threads within the same process share the same memory but run independently

Key Features of Multithreading
- A thread is the smallest unit of execution in Java.
- Threads share the same memory space but run independently.
- Java provides the Thread class and Runnable interface to create threads.
- Multithreading ensures better CPU utilization by executing tasks simultaneously.
- Synchronization is needed to prevent data inconsistency when threads share resources.

Benefits of Multithreading

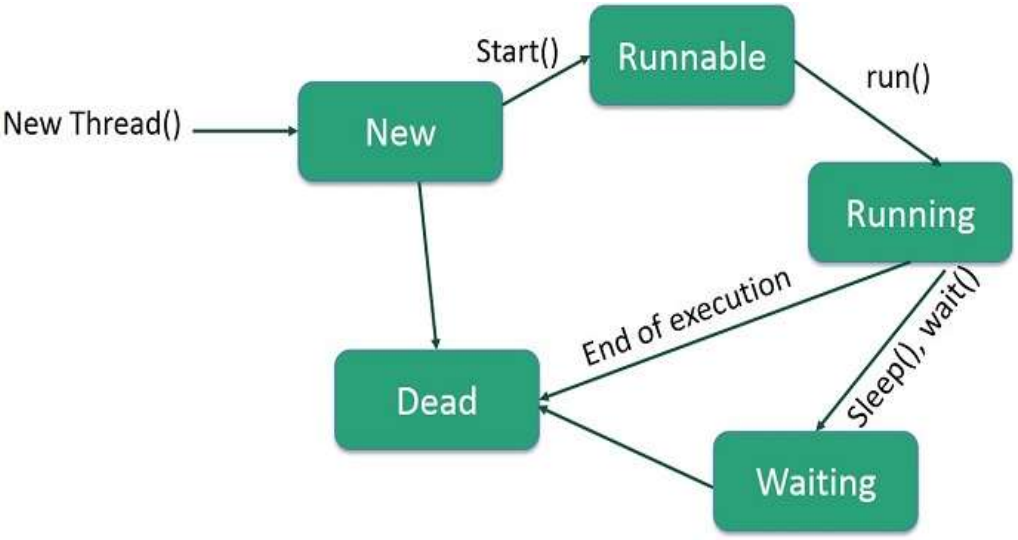
- Efficient use of CPU
- Faster execution
- Simultaneous tasks (e.g., playing music while downloading a file)
- Better resource sharing.

Thread vs Process

Aspect	Process	Thread
Memory	Has its own memory space	Shares memory with threads
Communication	Inter-process is complex	Easy communication
Overhead	High	Low

Thread Life-Cycle

NEW → RUNNABLE → RUNNING → WAITING/BLOCKED/SLEEPING → TERMINATED



States:

State	Description
New	Thread is created but not started.
Runnable	Thread is ready to run but waiting for CPU time.
Running	CPU has assigned time and thread is executing.
Blocked	Waiting to acquire a lock/synchronization.
Waiting	Waiting indefinitely until another thread notifies.
Timed Waiting	Sleeping or waiting for a specified time.
Terminated	Thread is finished or exited due to an error.

Ways to Create Threads.

1.By Extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // starts a new thread
    }
}
```

2.By Implementing Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start();
    }
}
```

Common Thread Methods – With Example

Method	Purpose
start()	Starts new thread
run()	Code executed by thread
sleep(ms)	Pauses thread for given time
join()	Waits for thread to finish
yield()	Gives chance to other threads
isAlive()	Checks if thread is running
setName() / getName()	Thread naming
setPriority() / getPriority()	Controls thread priority
getId()	Returns unique thread ID
getState()	Returns current state
currentThread()	Returns currently executing thread

Example: To domenstate yield() and Join()

```
class MyThread extends Thread {

    public MyThread(String name) {
        setName(name);
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " → " + i);

            // Use yield() inside the loop
```

```
            Thread.yield();  // Give chance to other threads
        }
    }
}

public class YieldJoinDemo {
    public static void main(String[] args) throws Exception {

        MyThread t1 = new MyThread("Thread-1");
        MyThread t2 = new MyThread("Thread-2");

        t1.start();
        t2.start();

        // join() -> main thread waits for both t1 and t2 to finish
        t1.join();
        t2.join();

        System.out.println("Main thread ends after both threads complete.");
    }
}
```

Example : To Demonstrate all important methods in Thread

```
class MyThread extends Thread {

    public MyThread(String name) {
        setName(name); // Setting thread name
    }

    public void run() {
        System.out.println(getName() + " started. | ID: " + getId());

        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " → Count: " + i);

            Thread.yield(); // Asking scheduler to give chance to other threads

            try {
                Thread.sleep(500); // TIMED_WAITING
            } catch (InterruptedException e) {
                System.out.println(getName() + " interrupted.");
            }
        }

        System.out.println(getName() + " finished.");
    }
}

public class ThreadMethodsDemo {

    public static void main(String[] args) throws Exception {

        System.out.println("Main Thread: " + Thread.currentThread().getName());
```

```
        System.out.println("Main Thread State: " + Thread.currentThread().getState());

        MyThread t1 = new MyThread("Worker-1");
        MyThread t2 = new MyThread("Worker-2");

        // Setting Priority (1–10)
        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.MAX_PRIORITY); // 10

        System.out.println("t1 Priority: " + t1.getPriority());
        System.out.println("t2 Priority: " + t2.getPriority());

        System.out.println("Before Start → t1 Alive? " + t1.isAlive());

        t1.start();  // RUNNABLE
        t2.start();

        System.out.println("After Start → t1 Alive? " + t1.isAlive());

        // Checking state
        System.out.println("t1 State: " + t1.getState());
        System.out.println("t2 State: " + t2.getState());

        // join() → main thread waits until t1 and t2 finish
        t1.join();
        t2.join();

        System.out.println("After join → t1 Alive? " + t1.isAlive());
        System.out.println("Main Thread Finished.");
    }
}
```

What is a Thread Scheduler?

A Thread Scheduler is part of the Java Virtual Machine (JVM) responsible for deciding which thread should run at a given time.

When is it used?

When multiple threads are in the “Runnable” state, the scheduler picks one to execute.

Thread Priority & Scheduler

Each thread has a priority between 1 and 10:

Constant	Value
Thread.MIN_PRIORITY	1
Thread.NORM_PRIORITY	5
Thread.MAX_PRIORITY	10

Example – Thread Scheduler

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Running thread: " +
Thread.currentThread().getName());
   }
}

public class SchedulerDemo {
   public static void main(String[] args) {
      MyThread t1 = new MyThread();
      MyThread t2 = new MyThread();

      t1.setPriority(Thread.MIN_PRIORITY); // 1
      t2.setPriority(Thread.MAX_PRIORITY); // 10

      t1.start();
      t2.start();
   }
}
```

Example with sleep() and join():

```
class MyThread extends Thread {
   public void run() {
      for(int i = 1; i <= 3; i++) {
         System.out.println("Running: " + i);
         try { Thread.sleep(1000); } catch (InterruptedException e) {}
      }
   }
}

public class Main {
   public static void main(String[] args) throws InterruptedException {
      MyThread t = new MyThread();
      t.start();
      t.join(); // main waits for t to finish
      System.out.println("Main thread ends.");
   }
}
```

Synchronization

Multithreading means executing multiple threads simultaneously within a single program.Threads share same memory, so they can access common data.This can improve performance but may cause data inconsistency if not handled properly.

What is a Race Condition?

A race condition occurs when:

- Two or more threads access shared data
- And try to modify it at the same time
- The final result depends on which thread executes first

Example Scenario

- Two threads try to increment the same variable
- Both read the old value before either writes back
- Result becomes incorrect

Example of Race Condition (Without Synchronization)

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}

class MyThread extends Thread {
    Counter c;

    MyThread(Counter c) {
        this.c = c;
    }

    public void run() {
        for(int i = 1; i <= 1000; i++) {
            c.increment();
        }
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        MyThread t1 = new MyThread(c);
```

```
        MyThread t2 = new MyThread(c);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Count: " + c.count);
    }
}
```

Expected O/P : Final Count: 2000
Actual O/P : Final Count: 1783 / 1950 / 1992 (It may vary)

This inconsistency happens due to **race condition**.

What is Thread Synchronization?

Thread Synchronization is a mechanism to:

- Control access to shared resources
- Allow only one thread at a time to access critical code
- Prevent race condition

Critical Section

- A critical section is a part of code where shared data is accessed.
- This section must be protected using synchronization.

Synchronization Using synchronized Keyword

a) Synchronized Method

```
class Counter {
  int count = 0;

  synchronized void increment() {   //Only one thread executing increment() at one time
     count++;
  }
}
```

b) Synchronized Block

```
void increment() {
   synchronized(this) {  // Synchronizes only the required code block Better performance
than Synchronized method
      count++;
  } }
```

Example With Synchronization (Fixed Race Condition)

```
class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}
class MyThread extends Thread {
    Counter c;

    MyThread(Counter c) {
        this.c = c;
    }

    public void run() {
        for(int i = 1; i <= 1000; i++) {
            c.increment();
        }
    }
}
public class SynchronizationDemo {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        MyThread t1 = new MyThread(c);
        MyThread t2 = new MyThread(c);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Final Count: " + c.count);
    }
}
```

How Synchronization Works Internally

- Every Java object has a **lock (monitor)**
- synchronized acquires the lock
- Other threads must wait until the lock is released

Types of Synchronization in Java

a) Object Level Synchronization

- Uses object lock
- Common for instance methods

```
synchronized void method() { }
```

b) Class Level Synchronization

- Uses class lock
- For static methods

```
static synchronized void method() { }
```

Advantages of Synchronization

- Prevents race condition
- Ensures data consistency
- Safe multithreaded execution

Disadvantages of Synchronization

- Slower performance
- Thread waiting (blocking)
- Can cause **deadlock** if misused

Example : Producer–Consumer using wait() and notify()

```
class SharedBuffer {
   private int data;
   private boolean hasData = false;

   // Producer method
   synchronized void produce(int value) {
      try {
         if (hasData) {
            wait();   // wait if data already present
         }
         data = value;
         System.out.println("Produced: " + data);
         hasData = true;
         notify();    // notify consumer
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }

   // Consumer method
   synchronized void consume() {
      try {
         if (!hasData) {
            wait();   // wait if no data available
         }
         System.out.println("Consumed: " + data);
         hasData = false;
         notify();    // notify producer
      } catch (InterruptedException e) {
         System.out.println(e);
      }
```

```
        }
    }
    // Producer Thread
    class Producer extends Thread {
        SharedBuffer buffer;
        Producer(SharedBuffer buffer) {
            this.buffer = buffer;
        }
        public void run() {
            for (int i = 1; i <= 5; i++) {
                buffer.produce(i);
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    System.out.println(e);
                }
            }
        }
    }
    // Consumer Thread
    class Consumer extends Thread {
        SharedBuffer buffer;
        Consumer(SharedBuffer buffer) {
            this.buffer = buffer;
        }
        public void run() {
            for (int i = 1; i <= 5; i++) {
                buffer.consume();
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    System.out.println(e);
                }
            }
        }
    }
    // Main Class
    public class ProducerConsumerDemo {
        public static void main(String[] args) {
            SharedBuffer buffer = new SharedBuffer();

            Producer p = new Producer(buffer);
            Consumer c = new Consumer(buffer);

            p.start();
            c.start();
        }
    }
```

Exercise :

SET A

1) Write a java program to Create a thread by extending the Thread class that prints numbers from 1 to 10.

2) Write a java program to Create a thread by implementing the Runnable interface that prints a message 5 times.

3) Write a java program to Create two threads using the Runnable interface. One thread prints even numbers, another prints odd numbers from 1 to 20.

4) Write a Java program to demonstrate the use of Thread.sleep() in a user-defined thread. Show how the main thread and the child thread execute concurrently with delayed output from the child thread.

SET B

1) Write a Java program by extending the Thread class to print numbers using two threads: one custom thread and the main thread. Use Thread.yield() to demonstrate its impact on thread scheduling and observe interleaved output.

2) Write a Java program to create multiple threads and assign names. Set different priorities for the threads. Display the thread name and priority during execution and pause the execution of each thread using the sleep() method.

3) Write a Java program to demonstrate the Producer–Consumer problem using wait**()** and notify**()** methods for inter-thread communication.
Note: Create a shared buffer accessed by two threads: a Producer thread that produces data and a Consumer thread that consumes the data.

4) Write a Java program to simulate a ticket booking system where multiple threads try to book tickets from a limited number of available tickets. Use synchronization to prevent race conditions and ensure correct ticket allocation.

SET C

1) Write a Java program to simulate a bank account where multiple threads perform deposit and withdrawal operations concurrently. Ensure thread safety using synchronization.

2) Write a Java program to simulate an office print management system where multiple employees send print requests simultaneously. Each print request should be treated as a task and each printer should act as a thread. Use a fixed-size thread pool of three threads to efficiently process all print jobs by reusing the available printers..

Name and Signature of the Instructor: _____

Date of Evaluation: _____

Assignment Evaluation:

0. Not Done: ☐ 1. Incomplete: ☐ 2. Late Complete: ☐

3. Needs Improvement : ☐ 4. Complete : ☐ 5. Well Done : ☐

Assignment No 3 : JDBC

What is JDBC?

JDBC is a Java API that allows Java programs to connect and interact with databases.

JDBC = Java + Database Communication

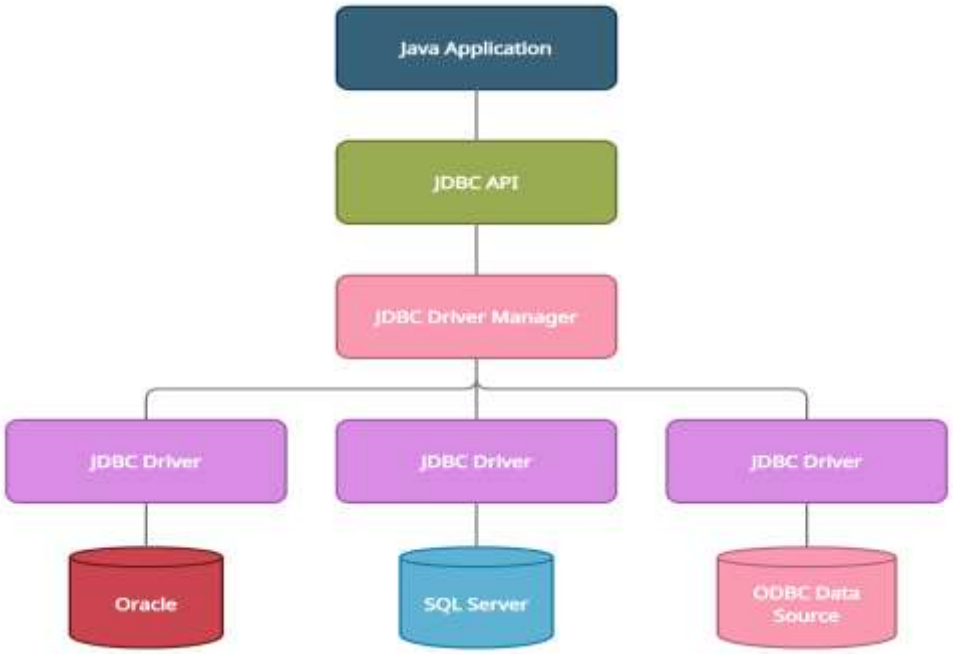
JDBC helps perform:

- Insert
- Update
- Delete
- Retrieve (Select)

JDBC Architecture

- **JDBC API** – Interfaces like Connection, Statement, etc.

- **JDBC Driver** – Actual implementation provided by DB vendor (e.g., MySQL JDBC driver)



Steps to Connect Java with Database using JDBC

Key Steps:

Step	Description
1	Load the Driver class
2	Create a Connection
3	Create a Statement
4	Execute the Query
5	Close the Connection

Example : To set data into database using statement

```
import java.sql.*;
import java.util.Scanner;

class StudentInsertStatement {
public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter Roll: ");
int roll = sc.nextInt();
sc.nextLine();

System.out.print("Enter Name: ");
String name = sc.nextLine();

System.out.print("Enter City: ");
String city = sc.nextLine();

String url = "jdbc:postgresql://localhost:5432/testdb";
String user = "postgres";
String password = "postgres";

try {
// 1. Load Driver
Class.forName("org.postgresql.Driver");

// 2. Create Connection
Connection con = DriverManager.getConnection(url, user,
    password);

// 3. Create Statement
Statement stmt = con.createStatement();

// 4. SQL Query (String Concatenation)
String sql = "INSERT INTO student VALUES ("
+ roll + ", '"
```

```
        + name + "', '"
        + city + "')";

        // 5. Execute
        int result = stmt.executeUpdate(sql);

        if (result > 0)
        System.out.println("Student record inserted successfully");

        // 6. Close resources
        stmt.close();
        con.close();

        } catch (Exception e) {
        e.printStackTrace();
        }
        }
}
```

Example to set data into database using prepare statement

```
import java.sql.*;
import java.util.Scanner;

class StudentInsert {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Roll: ");
        int roll = sc.nextInt();
        sc.nextLine();   // consume newline

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter City: ");
        String city = sc.nextLine();

        String url =
      "jdbc:postgresql://localhost:5432/testdb";
        String user = "postgres";
        String password = "postgres";

        try {
            // 1. Load Driver
            Class.forName("org.postgresql.Driver");

            // 2. Create Connection
            Connection con =
```

```
                    DriverManager.getConnection(url, user, password);

                        // 3. SQL Query
                        String sql = "INSERT INTO student VALUES
                    (?, ?, ?)";

                        // 4. PreparedStatement
                        PreparedStatement ps =
                    con.prepareStatement(sql);
                        ps.setInt(1, roll);
                        ps.setString(2, name);
                        ps.setString(3, city);

                        // 5. Execute
                        int result = ps.executeUpdate();

                        if (result > 0)
                            System.out.println("Student record inserted
                    successfully");

                        // 6. Close resources
                        ps.close();
                        con.close();

                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }
}
```

Example with MySQL to get data from db

```
import java.sql.*;
public class JDBCExample {
   public static void main(String[] args) {
      try {
         // 1. Load the driver
         Class.forName("com.mysql.cj.jdbc.Driver");
         // 2. Establish connection
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb", "root", "password");
         // 3. Create statement
         Statement stmt = con.createStatement();
         // 4. Execute query
         ResultSet rs = stmt.executeQuery("SELECT * FROM students");
         // 5. Process the result
         while (rs.next()) {
            System.out.println(rs.getInt(1) + " " + rs.getString(2));
         }
         // 6. Close connection
         con.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
```

Example with Psql(Using PostgreSQL)

```
import java.sql.*;

public class PostgresJDBCExample {
   public static void main(String[] args) {
      // Database credentials
      String url = "jdbc:postgresql://localhost:5432/college";
      String user = "postgres";
      String password = "your_password";  // Replace with your actual
    password

      try {
         // 1. Load PostgreSQL JDBC Driver (optional since JDBC 4.0)
         Class.forName("org.postgresql.Driver");

         // 2. Establish the connection
         Connection con = DriverManager.getConnection(url, user,
    password);
         System.out.println("Connected to PostgreSQL!");

         // 3. Create Statement
         Statement stmt = con.createStatement();
```

```
            // 4. Execute a query (SELECT example)
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");

            // 5. Process the ResultSet
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                int age = rs.getInt("age");

                System.out.println(id + " | " + name + " | " + age);
            }

            // 6. Close resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Important JDBC Classes and Interfaces

Interface / Class	Description
DriverManager	Manages DB drivers and connections
Connection	Connects Java to DB
Statement	Used to run simple SQL queries
PreparedStatement	Used for parameterized queries
ResultSet	Stores result from SELECT query

Execute Methods

Method	Used for
executeQuery()	SELECT – returns ResultSet
executeUpdate()	INSERT, UPDATE, DELETE – returns int (rows affected)
execute()	For unknown query type (returns boolean)

ResultSet Methods

Method	Description
next()	Moves to next row

Method	Description
getInt(1)	Gets int from column 1
getString("name")	Gets string from column "name"

Closing Resources

Always close resources in this order:

- rs.close();
- stmt.close();
- con.close();

Types of JDBC Drivers

Type	Description	Example
Type 1	JDBC-ODBC Bridge	Deprecated
Type 2	Native-API Driver	Oracle OCI
Type 3	Network Protocol Driver	App server
Type 4	Thin Driver (Pure Java)	MySQL, Oracle JDBC drivers

Example: Create table, insert record by user and display

```
import java.sql.*;
import java.util.Scanner;

public class EmployeeJDBCUserInput {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/test";
        String user = "root";
        String password = "root";

        Scanner sc = new Scanner(System.in);

        try {
            // 1. Load Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Create Connection
            Connection con = DriverManager.getConnection(url, user, password);

            // 3. Create Statement
            Statement stmt = con.createStatement();
```

```
            // 4. Create Table
            stmt.execute("CREATE TABLE IF NOT EXISTS employee (" +
                    "id INT PRIMARY KEY, " +
                    "name VARCHAR(50), " +
                    "mobile VARCHAR(15))");

            // 5. Take Input from User
            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine(); // consume newline

            System.out.print("Enter Employee Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Mobile Number: ");
            String mobile = sc.nextLine();

            // 6. Insert Record
            String insertQuery = "INSERT INTO employee VALUES (" +
                    id + ", '" + name + "', '" + mobile + "')";

            stmt.executeUpdate(insertQuery);
            System.out.println("Employee record inserted");

            // 7. Display Records
            ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

            System.out.println("\nEmployee Details:");
            System.out.println("ID\tName\tMobile");

            while (rs.next()) {
               System.out.println(
                       rs.getInt("id") + "\t" +
                       rs.getString("name") + "\t" +
                       rs.getString("mobile")
               );
            }

            // 8. Close Connection
            con.close();
            sc.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Example :

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class SimpleJDBCPostgres {
   public static void main(String[] args) {
      String url = "jdbc:postgresql://localhost:5432/testdb";
      String user = "username";
      String password = "password";

      try {
         // Load PostgreSQL JDBC Driver (optional in newer JDKs)
         Class.forName("org.postgresql.Driver");

         // Create connection
         Connection con = DriverManager.getConnection(url, user, password);
         System.out.println("Connected to PostgreSQL!");

         // Create statement and execute query
         Statement stmt = con.createStatement();
         ResultSet rs = stmt.executeQuery("SELECT * FROM students");

         // Print result
         while (rs.next()) {
            System.out.println(rs.getInt("id") + " | " +
                               rs.getString("name") + " | " +
                               rs.getInt("age"));
         }

         // Close resources
         rs.close();
         stmt.close();
         con.close();

      } catch (Exception e) {
         e.printStackTrace();
      }
   }
}
```

Exercise:

SET A

1) Write a program to connect to a database and display a success message.
2) Create a table Student(id, name, marks) using JDBC and insert some records.
3) Write a program to display all student records from the table using ResultSet

SET B

4) Use PreparedStatement to insert data into the student table using user input.
5) Write a Java program using JDBC to retrieve and display database information such as database product name, database version, driver name, and user name using the DatabaseMetaData interface..
6) Design **a** menu-driven Java application using JDBC to manage student records stored in a database.
   The program should allow the user to insert, display, search, update, and delete student information using JDBC connectivity.

SET C

7) Write a Java program to fetch all employee records from a table.
   Use ResultSetMetaData to dynamically display column names and their data types along with the data. t.
8) Display all column names and table metadata using DatabaseMetaData and ResultSetMetaData.

Name and Signature of the Instructor: _____

Date of Evaluation: _____

Assignment Evaluation:

0. Not Done: ☐ 1. Incomplete : ☐ 2. Late Complete : ☐

3.Needs Improvement : ☐ 4. Complete : ☐ 5. Well Done : ☐

Assignment No 4 : Servlet JSP

Servlet

- A Java class used to handle HTTP requests and generate dynamic content (usually HTML) on a web server.
- Acts as a controller in MVC architecture.
- Runs on the server inside a Servlet Container (like Apache Tomcat).

A Servlet is:

- A Java class.
- Runs on a server (like Apache Tomcat).
- Handles requests (usually HTTP) from a browser.
- Sends back a response (like HTML).

Responsibilities of a Servlet:

- Read form data sent by the user
- Process data (e.g., calculations, DB query)
- Generate dynamic web pages (can forward to JSP)

JSP (JavaServer Pages)

- An HTML page with embedded Java code using special tags (`<% %>`).
- Used mainly to present the response (View in MVC).
- Easier to write and maintain than Servlets for designing UI.

A JSP is:

- A mix of HTML and Java code**.**
- Easier to write UI because it looks like a webpage.
- Automatically converted into a Servlet by the server.
- Used to display results to the user.

How JSP and Servlets Work Together?

- Client (browser) → sends request
- Servlet processes → business logic
- Servlet forwards data to JSP
- JSP generates HTML response back to client.

Directory Structure in Web App (Servlet + JSP)

```
MyWebApp/
├── WEB-INF/
│   ├── web.xml
│   └── classes/
│       └── MyServlet.class
└── index.jsp
```

web.xml (Deployment Descriptor)

```
<web-app>
        <servlet>
          <servlet-name>MyServlet</servlet-name>
          <servlet-class>MyServlet</servlet-class>
        </servlet>
        <servlet-mapping>
          <servlet-name>MyServlet</servlet-name>
          <url-pattern>/hello</url-pattern>
        </servlet-mapping>
</web-app>
```

Simple Servlet Example(MyServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class MyServlet extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException, ServletException {
      res.setContentType("text/html");
      PrintWriter out = res.getWriter();

      String name = req.getParameter("name");

      // Forwarding data to JSP
```

```
        req.setAttribute("userName", name);
        RequestDispatcher rd = req.getRequestDispatcher("welcome.jsp");
        rd.forward(req, res);
    }
}
```

Simple JSP Example

```
<%@ page language="java" contentType="text/html" %>
                        <html>
                        <head><title>Welcome</title></head>
                        <body>
                          <h2>Welcome, <%= request.getAttribute("userName") %>!</h2>
                        </body>
</html>
```

How It Runs

1. User visits http://localhost:8080/MyWebApp/hello?name=yourname
2. Servlet receives name = yourname
3. Sets attribute "userName" = yourname
4. Forwards request to welcome.jsp
5. JSP prints: Welcome, yourname**!**

Common JSP Tags

Tag Type	Syntax	Purpose
Scriptlet	<% code %>	Java logic
Expression	<%= value %>	Output to browser
Declaration	<%! %>	Declare variables/methods
Directive	<%@ %>	Import, page settings

Servlet vs JSP

Feature	Servlet	JSP
Code Style	Pure Java	HTML + Java
Good For	Logic/Controller	UI/View
Compilation	Written by user	Auto converted to Servlet
Reusability	Less UI reusability	Easier to reuse HTML components
Maintenance	Harder for UI	Easier to update UI

Servlet + JSP = MVC Pattern

Component	Role	Example
Model	Data/Logic	Java Beans / DB
View	UI	JSP
Controller	Control Flow	Servlet

JSP vs Servlet (Summary)

Feature	JSP	Servlet
Used for	View/UI	Controller/Logic
Looks like	HTML + Java	Pure Java
Easier for	Designers	Developers
Compiled to	Servlet internally	Java class

Example : Simple Servlet to Display a Message

Step 1: Create Servlet Class

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");
        out.println("<h2>Hello, Welcome to Servlet!</h2>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Step 2: Run on Server

 Create Dynamic Web Project
 Add this servlet class
 Run on Apache Tomcat
 Open browser and type

```
http://localhost:8080/YourProjectName/hello
```

Example with HTML Form (Very Common in Lab)

```
<html>
<body>
   <form action="welcome" method="post">
      Enter Name:
      <input type="text" name="uname">
      <input type="submit" value="Submit">
   </form>
</body>
</html>
```

WelcomeServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      String name = request.getParameter("uname");

      out.println("<h2>Welcome, " + name + "</h2>");
   }
}
```

Example: Servlet + JDBC

Database Table (MySQL)

```
CREATE TABLE student (
    roll INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50)
);
```

HTML Form (student.html)

```
<html>
<body>
    <h2>Add Student</h2>
    <form action="addStudent" method="post">
        Roll No: <input type="text" name="roll"><br><br>
        Name: <input type="text" name="name"><br><br>
        City: <input type="text" name="city"><br><br>
        <input type="submit" value="Save">
    </form>
</body>
</html>
```

Servlet Code (AddStudentServlet.java)

```
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/addStudent")
public class AddStudentServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        int roll = Integer.parseInt(request.getParameter("roll"));
        String name = request.getParameter("name");
        String city = request.getParameter("city");
```

```
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test", "root", "root");

            Statement stmt = con.createStatement();

            String query = "INSERT INTO student VALUES (" +
                  roll + ",'" + name + "','" + city + "')";

            stmt.executeUpdate(query);

            out.println("<h3>Student Added Successfully</h3>");

            con.close();
        } catch (Exception e) {
            out.println(e);
        }
    }
}
```

Exercise:

SET A

1. Write a Program to create a simple servlet that prints “Welcome to Java Servlet”.
2. Write a Program to Create a JSP page that displays “Hello User!” using scriptlet and expression tag.
3. Write a servlet to accept user name as input via URL parameters and display a greeting.

SET B

1. Create a servlet to accept user input (name, age) using HTML form and display it using GET method.
2. Create a JSP page that accepts two numbers and displays their sum using scripting elements.
3. Write a servlet to forward a request to another servlet and include content using RequestDispatcher.
4. Write a servlet to create a cookie and send it to the browser, then read and display cookies received from the browser.

SET C

1. Demonstrate session tracking using HTTP Session by creating login and welcome pages using Servlet and JSP.

2. Develop a web application using Servlet and JDBC to perform insert, display, update, and delete operations on student records stored in a database

3. Create a Student Registration System using JSP and Servlet that accepts student details, stores them in a database, and displays a success message.

Name and Signature of the Instructor: _____

Date of Evaluation: _____

Assignment Evaluation:

0. Not Done: ☐ 1. Incomplete: ☐ 2. Late Complete: ☐

3. Needs Improvement: ☐ 4. Complete: ☐ 5. Well Done: ☐

Section II

CSMJ362
Data Analytics

Assignment 1: Linear and Logistic Regression

No. of slots: 02

Objectives

- Apply appropriate analytic techniques and tools to analyze data, create models, and identify insights that can lead to actionable results.
- Apply modeling and data analysis linear and logistic regression techniques to the solution of real world business problems

Reading

You should read the following topics before starting this exercise

- The modeling process, Engineering features and selecting a model, Training the model, Validating the model, Predicting new observations
- Types of machine learning
- Regression models
- Concept of classification, clustering and reinforcement learning

Ready Reference and Self Activity

Machine Learning -

- Machine Learning is said as a subset of **artificial intelligence** that is mainly concerned with the development of algorithms which allow a computer to learn from the data and past experiences on their own. The term machine learning was first introduced by **Arthur Samuel** in **1959**.
- Definition: Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.
- With the help of sample historical data, which is known as **training data**, machine learning algorithms build a **mathematical model** that helps in making predictions or decisions without being explicitly programmed. Machine learning brings computer science and statistics together for creating predictive models.

Machine learning can be classified into three types:

1. Supervised learning 2. Unsupervised learning 3. Reinforcement learning

1) Supervised Learning - Supervised learning is a type of machine learning method in which we provide sample labeled data to the machine learning system in order to train it, and on that basis, it predicts the output.

2) Unsupervised Learning - Unsupervised learning is a learning method in which a machine learns without any supervision.

3) Reinforcement Learning - Reinforcement learning is a feedback-based learning method, in which a learning agent gets a reward for each right action and gets a penalty for each wrong action. The agent learns automatically with these feedbacks and improves its performance. In reinforcement learning, the agent interacts with the environment and explores it. The goal of an agent is to get the most reward points, and hence, it improves its performance.

Regression Analysis-

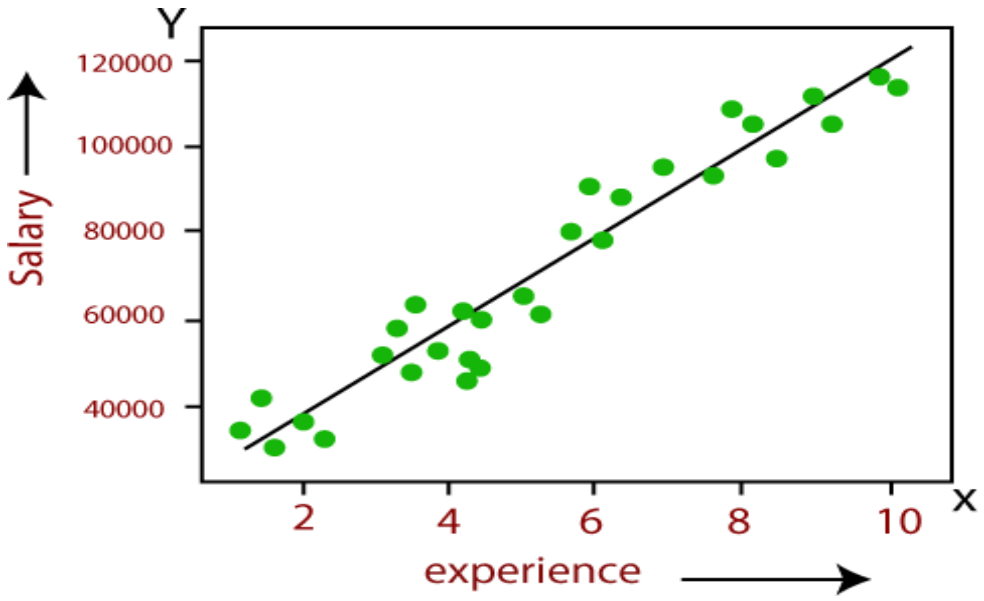
- Regression analysis is a statistical method to model the relationship between a dependent (target) and independent (predictor) variables with one or more independent variables.
- Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed.
- Regression is a supervised learning technique which helps in finding the correlation between variables and enables us to predict the continuous output variable based on the one or more predictor variables.
- It is mainly used for **prediction, forecasting, time series modeling, and determining the causal-effect relationship between variables**.
- In Regression, we plot a graph between the variables which best fits the given datapoints, using this plot, the machine learning model can make predictions about the data.
- ***"Regression shows a line or curve that passes through all the datapoints on target-predictor graph in such a way that the vertical distance between the datapoints and the regression line is minimum."***
- The distance between datapoints and line tells whether a model has captured a strong relationship or not.

Types of Regression

There are various types of regressions which are used in data science and machine learning. Each type has its own importance on different scenarios, but at the core, all the regression methods analyze the effect of the independent variable on dependent variables. Here in this assignment we will learn Linear Regression and Logistic Regression in detail.

Linear Regression:

- Linear regression is a statistical regression method which is used for predictive analysis.
- It is one of the very simple and easy algorithms which works on regression and shows the relationship between the continuous variables.
- It is used for solving the regression problem in machine learning.
- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- If there is only one input variable (x), then such linear regression is called **simple linear regression**. And if there is more than one input variable, then such linear regression is called **multiple linear regression**.
- The relationship between variables in the linear regression model can be explained using the below image. Here we are predicting the salary of an employee on the basis of **the year of experience**.



- Below is the mathematical equation for Linear regression:
    - Y= aX+b
        - **Here,**
            - **Y = dependent variables (target variables),**
            - **X= Independent variables (predictor variables),**
            - **a and b are the linear coefficients**

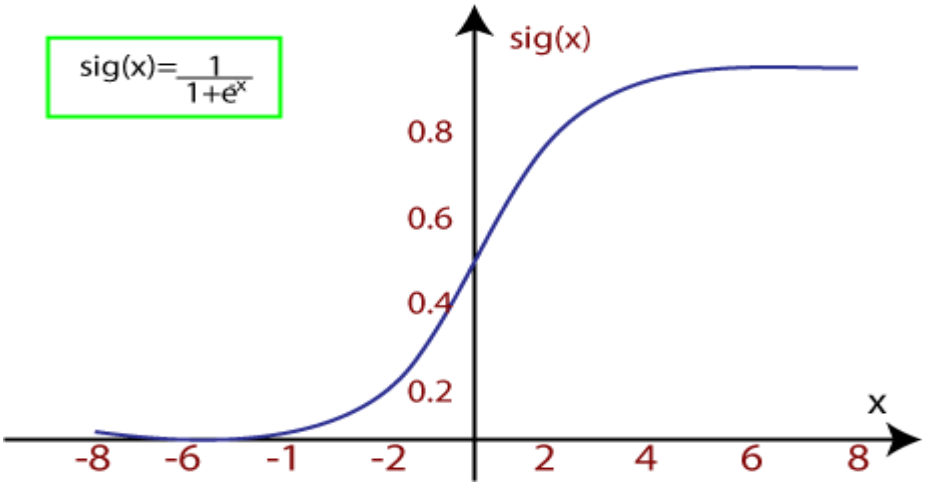
Logistic Regression:

- Logistic regression is another supervised learning algorithm which is used to solve the classification problems. In **classification problems**, we have dependent variables in a binary or discrete format such as 0 or 1.
- Logistic regression algorithm works with the categorical variable such as 0 or 1, Yes or No, True or False, Spam or not spam, etc.
- It is a predictive analysis algorithm which works on the concept of probability.
- Logistic regression is a type of regression, but it is different from the linear regression algorithm in the term how they are used.
- Logistic regression uses **sigmoid function** or logistic function which is a complex cost function. This sigmoid function is used to model the data in logistic regression. The function can be represented as:

$$f(x)= \frac{1}{1+e^{-x}}$$

    - f(x)= Output between the 0 and 1 value.
    - x= input to the function
    - e= base of natural logarithm.

When we provide the input values (data) to the function, it gives the S-curve as follows:



- It uses the concept of threshold levels, values above the threshold level are rounded up to 1, and values below the threshold level are rounded up to 0.

There are three types of logistic regression:

- **Binary**- In this type, the dependent/target variable has two distinct values, either 0 or 1, malignant or benign, passed or failed, admitted or not admitted.
- **Multinominal** - Multinomial Logistic Regression deals with cases when the target or independent variable has three or more possible values. (cats, dogs, lions)
- **Ordinal** – It is used in cases when the target variable is of ordinal nature. In this type, the categories are ordered in a meaningful manner and each category has quantitative significance. (low, medium, high)

Self-Activity

Building a linear regression model in Python

1. Import libraries /packages
2. Reading and understanding the data(eventually do appropriate transformations)
3. Visualizing the data
4. Splitting our Data set in Dependent and Independent variables.
5. Performing simple linear regression (create linear regression model)
6. Residual analysis(Check the results of model fitting to know whether the model is satisfactory)
7. Predictions on the test set (apply the model)

Sample Example -

Goal is to build a linear regression model in Python

For this example we are using car data. Following is the link to download the rquired dataset

https://www.kaggle.com/CooperUnion/cardataset

1. Import libraries /packages

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_squared_error
%matplotlib inline
```

2. Reading and understanding the data(eventually do appropriate transformations)

```
df = pd.read_csv('C:/TYBSC/car_data.csv') # Importing the data set
df.sample(5) #previewing dataset randomly
```

Then we import the car dataset. And print 5 sample dataset values. At first, we imported our necessary libraries.

```
print(df.shape) # view the dataset shape
print(df['Make'].value_counts()) # viewing Car companies with their cars number
```

Here we print the shape of the dataset and print the different car companies with their total cars.

```
new_df = df[df['Make']=='Volkswagen'] # in this new data set we only take 'Volkswagen' Cars
print(new_df.shape) # Viewing the new dataset shape
print(new_df.isnull().sum()) # Is there any Null or Empty cell presents
new_df = new_df.dropna() # Deleting the rows which have Empty cells
new_df.shape # After deletion Vewing the shape
new_df.isnull().sum() #Is there any Null or Empty cell presents
new_df.sample(2) # Checking the random dataset sample
```

Here we select only ‘Volkswagen’ cars from the large dataset. Because different types of cars have different brand value and higher or lower price. So we take only one car company for better prediction.
Then we view the shape and check if any null cell present or not. We found there are many null cells present. We delete those rows which have null cells. It is very important when you make a dataset for fitting any data model. Then we cross check if any null cells present or not. No null cell found then we print 5 sample dataset values.

```
new_df = new_df[['Engine HP','MSRP']] # We only take the 'Engine HP' and 'MSRP' columns
new_df.sample(5) # Checking the random dataset sample
```

Here we select only 2 specific (‘Engine HP’ and ‘MSRP’) columns from all columns. It is very important to select only those columns which could be helpful for prediction. It depends on your common sense to select those columns. Please select those columns that wouldn’t spoil your prediction. After select only 2 columns, we view our new dataset.

```
X = np.array(new_df[['Engine HP']]) # Storing into X the 'Engine HP' as np.array
y = np.array(new_df[['MSRP']]) # Storing into y the 'MSRP' as np.array
```

```
print(X.shape) # Vewing the shape of X
print(y.shape) # Vewing the shape of y
```

Here we put the **'Engine HP'** column as a numpy array into **'X'** variable. And **'MSRP'** column as a numpy array into **'y'** variable. Then check the shape of the array.

3. Visualizing the data

```
plt.scatter(X,y,color="red") # Plot a graph X
vs y plt.title('HP vs MSRP')
plt.xlabel('HP')
plt.ylabel('MSR
P') plt.show()
```

Here we plot a scatter plot graph between **'MSRP'** and **'HP'.** After viewing this graph we ensured that we can perform a linear regression for prediction.

4. Splitting our Data set in Dependent and Independent variables.

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size =
0.25,random_state=15) # Spliting into train & test dataset
```

Here we split our 'X' and 'y' dataset into 'X_train', 'X_test' and 'y_train', 'y_test'. Here we take 25% data as test dataset and remaining as train dataset. We take the random_state value as 15 for our better prediction.

5. Performing simple linear regression (create linear regression model)

```
regressor = LinearRegression() # Creating a regressior
regressor.fit(X_train,y_train) # Fiting the dataset into the
model
```

We create regressor. And we fit the X_train and y_train into the regressor model.

6. Residual analysis(Check the results of model fitting to know whether the model is satisfactory)

```
plt.scatter(X_test,y_test,color="green") # Plot a graph with X_test vs y_test
plt.plot(X_train,regressor.predict(X_train),color="red",linewidth=3) # Regressior line
showing plt.title('Regression(Test Set)')
plt.xlabel('HP')
plt.ylabel('MSR
P') plt.show()
```

Here we plot a scatter plot graph between X_test and y_test datasets and we draw a regression line.

```
plt.scatter(X_train,y_train,color="blue")  # Plot a graph with X_train vs y_train
plt.plot(X_train,regressor.predict(X_train),color="red",linewidth=3) # Regressior line
showing plt.title('Regression(training Set)')
plt.xlabel('HP')
plt.ylabel('MSRP')
plt.show()
```

Here we plot the final **X_train vs y_train** scatterplot graph with a **best-fit regression line**. Here we

can clearly understand the regression line.

7. Predictions on the test set (apply the model)

Here we print R^2,Mean Error, write function predict the price of car.

```
y_pred = regressor.predict(X_test)
print('R2 score: %.2f' % r2_score(y_test,y_pred)) # Priniting R^2 Score
print('Mean Error :',mean_squared_error(y_test,y_pred)) # Priniting the mean error
def car_price(hp): # A function to predict the price according to Horse power
    result = regressor.predict(np.array(hp).reshape(1, -1))
    return(result[0,0])
car_hp = int(input('Enter Volkswagen cars Horse Power : '))
print('This Volkswagen Price will be :
',int(car_price(car_hp))*69,'₹')
```

Linear regression example with Python code and scikit-learn

1. First, let's import linear_model from scikit-learn library: `from sklearn import linear_model`
2. Now take features and labels set to train our program: `features = [[2],[1],[5],[10]]` `labels = [27, 11, 75, 155]`
3. After that create our model and fit the label and features to our model: `clf = linear_model.LinearRegression()` `clf=clf.fit(features,labels)`
4. In the end, pass data to the model and print the predicted result: `predicted = clf.predict([[8]])` `print(predicted)`

Building a logistic regression model

Steps to build Logistic Regression model in Python:

1. Import libraries /packages
2. Reading and understanding the data(do appropriate transformations- cleaning, filling nulls, duplicates, etc…)
3. Splitting our Data set in Dependent and Independent variables.
4. Performing simple linear regression (create logistic regression model)
5. Print the Accuracy and plot the Confusion Matrix
6. Print test data and predicted data Predictions on the test set

Sample Example -

Goal is to build a logistic regression model in Python in order to determine whether candidates would get admitted to a prestigious university.

Here, there are two possible outcomes: **Admitted** (represented by the value of '1') vs. **Rejected** (represented by the value of '0').

You can then build a logistic regression in Python, where:

- The dependent variable represents whether a person gets admitted; and
- The 3 independent variables are the GMAT score, GPA and Years of work experience

1. Import libraries

```
/packages import
pandas as pd
import numpy as
np
from sklearn.model_selection import
train_test_split from sklearn.linear_model import
LogisticRegression from sklearn import metrics
import seaborn as sn
import matplotlib.pyplot as plt
```

2. Reading and understanding the data(eventually do appropriate transformations- cleaning, filling nulls, duplicates, etc…)

```
data = pd.read_csv("C:\TYBSC\Student_Score.csv")     # dataset
```

3. Splitting our Data set in Dependent and Independent variables.

In our Data set we'll consider **gmat** score, **gpa** and Years of **work_experience** as Independent variable and **admitted** as Dependent Variable.

```
x = dataset.iloc[:,
[2,3]].values y =
dataset.iloc[:, 4].values
```

Then, apply train_test_split. For example, you can set the test size to 0.25, and therefore the model testing will be based on 25% of the dataset, while the model training will be based on 75% of the dataset:

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=0)
```

4. Performing simple logistic regression (create regression model)

```
logistic_regression= LogisticRegression()
logistic_regression.fit(x_train,y_train)
y_pred=logistic_regression.predict(x_test)
```

5. Print the Accuracy and plot the Confusion

Matrix Then, use the code below to get the

Confusion Matrix:

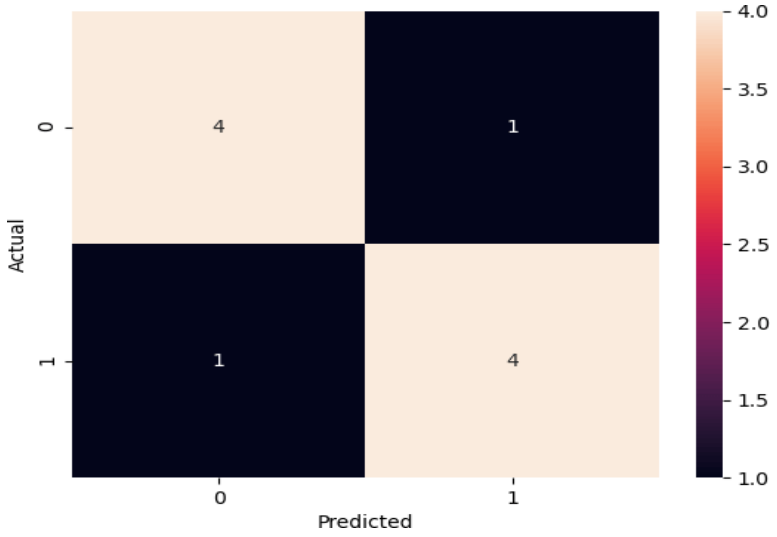
```
confusion_matrix = pd.crosstab(y_test, y_pred, rownames=['Actual'],
colnames=['Predicted']) sn.heatmap(confusion_matrix, annot=True)
```

For the final part, print the Accuracy and plot the Confusion Matrix:

```
print('Accuracy: ',metrics.accuracy_score(y_test, y_pred))
plt.show()
```

When we put all the code components together and Run the code in Python, will get the following Confusion Matrix with an Accuracy of 0.8

(note that depending on sklearn version, may get a different accuracy results. In above code case, the sklearn version is 0.22.2):



As can be observed from the matrix:

TP = True Positives = 4

TN = True Negatives = 4

FP = False Positives = 1

FN = False Negatives = 1

You can then also get the Accuracy using:

Accuracy = (TP+TN) / Total = (4+4) / 10 = 0.8 The accuracy is therefore 80% for the test set.

6. Print test data and predicted data Predictions on the test set

Diving Deeper into the Results -> print two components in the python code: print (x_test)

print (y_pred)

Recall that our original dataset (from step 1) had 40 observations. Since we set the test size to 0.25, then the confusion matrix displayed the results for 10 records (=40*0.25). These are the 10 test records:

	gmat	gpa	work_experience
22	550	2.3	4
20	620	3.3	2
25	670	3.3	6
4	680	3.9	4
10	610	2.7	3
15	610	3.0	1
28	650	3.7	6
11	690	3.7	5
18	540	2.7	2
29	660	3.3	5

The prediction was also made for those 10 records (where 1 = admitted, while 0 = rejected):

```
[0 0 1 1 0 0 1 1 0 1]
```

In the actual dataset (from step-1), you'll see that for the test data, we got the correct results 8 out of 10 times:

Index	gmat	gpa	work_experience	admitted - actual results	admitted - predicted results	Matching
22	550	2.3	4	0	0	TRUE
20	620	3.3	2	1	0	FALSE
25	670	3.3	6	1	1	TRUE
4	680	3.9	4	0	1	FALSE
10	610	2.7	3	0	0	TRUE
15	610	3	1	0	0	TRUE
28	650	3.7	6	1	1	TRUE
11	690	3.7	5	1	1	TRUE
18	540	2.7	2	0	0	TRUE
29	660	3.3	5	1	1	TRUE

This is matching with the accuracy level of 80%

Lab Assignments

SET A

1. Create 'sales' Data set having 5 columns namely: ID, TV, Radio, Newspaper and Sales.(random 500 entries) Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets. then divide the training and testing sets into a 7:3 ratio, respectively and print them. Build a simple linear regression model.
2. Create 'realestate' Data set having 4 columns namely: ID, flat, houses and purchases (random 500 entries). Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets and print them. Build a simple linear regression model for predicting purchases.
3. Create 'User' Data set having 5 columns namely: User ID, Gender, Age, EstimatedSalary and Purchased. Build a logistic regression model that can predict whether on the given parameter a person will buy a car or not.

SET B

1. Build a simple linear regression model for Fish Species Weight Prediction. (download dataset https://www.kaggle.com/aungpyaeap/fish-market?select=Fish.csv)
2. Use the iris dataset. Write a Python program to view some basic statistical details like percentile, mean, std etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris-virginica'. Apply logistic regression on the dataset to identify different species (setosa, versicolor, verginica) of Iris flowers given just 4 features: sepal and petal lengths and widths.. Find the accuracy of the model.

Signature of the instructor ☐ Date ☐

Assignment Evaluation

0: Not done ☐	2: Late Complete ☐	4: Complete ☐
1: Incomplete ☐	3: Needs improvement ☐	5: Well Done ☐

Assignment 2: Frequent itemset and Association rule mining

No. of slots: 02

Objectives

- To understand the impact of finding frequent patterns from large datasets.
- To learn the Apriori Algorithm which is used for frequent itemsets mining.
- To understand Association Rule Mining.
- To write and learn implementation of such concepts with Python.

Reading

You should read the following topics before starting this exercise:

- Why Pre-processing is must before analysis of data.
- What is support, confidence and lift.
- Learn definitions such as frequent itemsets, association between things, Apriori Property of sets.
- Basic understanding of libraries supported in Python for performing these tasks.

Ready Reference

Frequent Itemset Mining: Finding frequent patterns, associations, correlations, or causal structures among sets of items or objects in transaction databases, relational databases, and other information repositories.

Association Mining searches for frequent items in the data-set. In frequent mining usually the interesting associations and correlations between item sets in transactional and relational databases are found.
If there are 2 items X and Y purchased frequently then it is good to put them together in stores or provide some discount offer on one item on purchase of other item. This can really increase the sales. For example it is likely to find that if a customer buys Milk and bread he/she also buys Butter. So the association rule is [‘milk]^[‘bread’]=>[‘butter’].

Applications: Market Basket Analysis is one of the key techniques used by large retailers to uncover associations between item, catalog design, loss-leader analysis, clustering, classification, recommendation systems, etc.

Consider the following Transaction database:
D= {{butter, bread, milk, sugar};
{butter, flour, milk, sugar};
{butter, eggs, milk, salt};
{eggs};
{butter, flour, milk, salt, sugar}}

Question of interest: Which items are bought together frequently?

Apriori is an algorithm for frequent item set mining and association rule learning over relational databases. The name of the algorithm is based on the fact that the algorithm uses *prior knowledge* of frequent itemset properties. Apriori employs an iterative approach known as a *levelwise* search, where k-itemsets are used to explore (k+1)-itemsets

To construct association rules between items, the algorithm considers 3 important factors which are, support, confidence and lift. Each of these factors is explained as follows:

Support:
The support of item I is defined as the ratio between the number of transactions containing the item I by the total number of transactions expressed as :

$$support(I) = \frac{Number\ of\ transactions\ containing\ I}{Total\ number\ of\ transactions}$$

Confidence:
This is measured by the proportion of transactions with item I1, in which item I2 also appears.

$$confidence(I1 \rightarrow I2) = \frac{Number\ of\ transactions\ containing\ items\ I1\ and\ I2}{Total\ number\ of\ transactions\ containing\ I1}$$

Given that the item on the left hand side (**antecedent**) is purchased then the item on the right hand side(**consequent**) would also be purchased.

Lift:
Lift is the ratio between the confidence and support expressed as :

$$lift(I1 \rightarrow I2) = \frac{confidence(I1 \rightarrow I2)}{support(I2)}$$

Lift (antecedent => consequent) = 1 means that there is no correlation within the itemset, > 1 means that there is a positive correlation within the itemset, i.e., products in the itemset, antecedent, and consequent, are more likely to be bought together, < 1 means that there is a negative correlation within the itemset, i.e., products in itemset, antecedent, and consequent, are unlikely to be bought together.

The steps of the apriori algorithm can be given as:-

1. Define the minimum support and confidence for the association rule
2. Take all the subsets in the transactions with higher support than the minimum support
3. Take all the rules of these subsets with higher confidence than minimum confidence
4. Sort the association rules in the decreasing order of lift.
5. Visualize the rules along with confidence and support.

In this assignment you will analyze collections of market baskets and will determine frequent itemsets and association rules present in the collections.

Python libraries

Python has many libraries for apriori implementation.

i. Mlxtend (apriori)
ii. Apyori (apriori)
iii. pypi (efficient_apriori)

The apriori module from mlxtend library provides fast and efficient apriori implementation.

Usage:
apriori(df, min_support=0.5, use_colnames=False, max_len=None, verbose=0, low_memory=False)

Parameters

- df : One-Hot-Encoded DataFrame or DataFrame that has 0 and 1 or True and False as values
- min_support : Floating point value between 0 and 1 that indicates the minimum support required for an itemset to be selected.
  # of observation with item / total observation# of observation with item / total observation
- use_colnames : This allows to preserve column names for itemset making it more readable.
- max_len : Max length of itemset generated. If not set, all possible lengths are evaluated.
- verbose : Shows the number of iterations if >= 1 and low_memory is True. If =1 and low_memory is False , shows the number of combinations.
- low_memory :
- If True, uses an iterator to search for combinations above min_support. Note that while low_memory=True should only be used for large dataset if memory resources are limited, because this implementation is approx. 3–6x slower than the default.

The function returns a pandas DataFrame with columns ['support', 'itemsets'] of all itemsets that are >= min_support and < than max_len (if max_len is not None).

Mining Association Rules

Frequent if-then associations called association rules which consists of an antecedent (if) and a consequent (then). The following function returns the association rules from the frequent itemsets which satisfy the given metric threshold. Metric can be set to confidence, lift, support, leverage and conviction.

association_rules(frequent_items, metric='confidence', min_threshold=0.5, support_only=False)

Leverage computes the difference between the observed frequency of A and C appearing together and the frequency that would be expected if A and C were independent. A leverage value of 0 indicates independence.
A high conviction value means that the consequent is highly depending on the antecedent.

Self-Activity

Step 1: Install the libraries

```
pip install mlxtend
```

Step 2: Import the libraries

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
```

Step 3: Read the data, encode the data

Create the sample dataset

```
transactions = [['eggs', 'milk','bread'],
                ['eggs', 'apple'],
                ['milk', 'bread'],
                ['apple', 'milk'],
                ['milk', 'apple', 'bread']]
```

The dataset contains a set of transactions with a set of text items. This needs to be converted into numerical form to be analyzed. The label encoding process is used to convert textual labels into numeric form in order to prepare it to be used in a machine-readable form. We can transform it into the right format via the TransactionEncoder as follows:

```
from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transactions).transform(transactions)
df=pd.DataFrame(te_array, columns=te.columns_)
df
```

You should see something like this

	apple	bread	Eggs	milk
0	False	True	True	True
1	True	False	True	False
2	False	True	False	True
3	True	False	False	True
4	True	True	False	True

Step 4: Find the frequent itemsets

Generate frequent itemsets that have a support value of at least 50%. By default, apriori returns the column indices of the items, For better readability, we can set use_colnames=True to convert these integer values into the respective item names:

```
freq_items = apriori(df, min_support = 0.5, use_colnames = True)
print(freq_items)
```

You will get

	support	itemsets
0	0.6	(apple)
1	0.6	(bread)
2	0.8	(milk)
3	0.6	(milk, bread)

Change the value of the min_support and see the output

Step 5: Generate the association rules

Generate association rules that have a support value of at least 5%

```
rules = association_rules(freq_items, metric ='support', min_threshold=0.05
)
rules = rules.sort_values(['support', 'confidence'], ascending =[False,Fals
e])
print(rules)
```

The output will be:

```
  antecedents consequents  antecedent support  consequent support  support  \
1     (bread)      (milk)                 0.6                 0.8      0.6
0      (milk)     (bread)                 0.8                 0.6      0.6

   confidence  lift  leverage  conviction
1        1.00  1.25      0.12         inf
0        0.75  1.25      0.12         1.6
```

To perform the above on a standard dataset, apply the following steps:

1. Download the csv file

```
from google.colab import files
data = files.upload()
```

2. Read the data from the csv file

```
import io
import pandas as pd
df = pd.read_csv(io.BytesIO(data['Market_Basket_Optimisation.csv']))
```

3. View the contents, info, columns and other details
4. Now, Convert Pandas DataFrame into a list of lists for encoding

```
transactions = []
for i in range(0, len(df)):
    transactions.append([str(df.values[i,j]) for j in range(0, len(df.c
olumns))])
```

5. Apply TransformEncoder to the transactions list
6. Apply the apriori algorithm

Dataset Sources

https://www.kaggle.com/datasets/sivaram1987/association-rule-learningapriori
https://github.com/shivang98/Market-Basket-Optimization
https://www.kaggle.com/datasets/hemanthkumar05/market-basket-optimization
https://www.kaggle.com/datasets/irfanasrullah/groceries

Lab Assignments

SET A:

1. Create the following dataset in python

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Convert the categorical values into numeric format.

Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

2. Create your own transactions dataset and apply the above process on your dataset.

SET B:

1. Download the Market basket dataset.
   Write a python program to read the dataset and display its information.
   Preprocess the data (drop null values etc.)
   Convert the categorical values into numeric format.
   Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules.
2. Download the groceries dataset.
   Write a python program to read the dataset and display its information.
   Preprocess the data (drop null values etc.)
   Convert the categorical values into numeric format.
   Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules.

SET C:

Write a python code to implement the apriori algorithm. Test the code on any standard dataset.

Signature of the instructor ☐ Date ☐

Assignment Evaluation

0: Not done ☐	2: Late Complete ☐	4: Complete ☐
1: Incomplete ☐	3: Needs improvement ☐	5: Well Done ☐

Assignment 3 : Text and Social Media Analytics

No. of slots: 03

Objectives

- To understand the concept of sentiment analysis.
- To learn various methodologies for analysis on text including text analytics, tokenization, frequency distribution, stopwords, stemming, lemmatization, part-of-speech tagging.
- To write the Python scripts using various libraries for sentiment analysis using natural language processing toolkit and classifying emotions on basis of labels i.e. Positive, Negative and Neutral. Also to use wordcloud package for words comparison.
- To perform analysis on social media data such as Facebook, Twitter, YouTube.
- To graphically represent the analyzed data.

Reading

You should read the following topics before starting the exercise :
What is the need of doing data analysis using natural language processing. Basics of Python libraries such as pandas, matplotlib, numpy, scikit-learn, nltk, VADER tool to perform the data analysis.

Ready Reference

Python Libraries for performing text and Sentiment Analysis :

Natural Language Toolkit (NLTK) :

NLTK is a Python Package for performing Natural Language Processing on human language data which is mostly unstructured. It mainly focuses on analyzing textual data. It supports different natural language processing algorithms such as Tokenization, Frequency Distribution, Stopwords, Lexicon Normalization, Stemming, Lemmatization, POS Tagging. These are considered as pre-processing steps to perform text analytics.

Installation of NLTK : You can use any IDE to perform Python programming for the following tasks. Here Spyder IDE is used.

✓ To install NLTK, use pip as follows :

```
pip install nltk
```

✓ Then download the supportable NLTK packages using :

```
import nltk
nltk.download()
```

✓ After running the above script, a screen will come to download the packages. Here click on download to download all the supporting NLTK packages.

You can also download all NLTK packages using Python statement :

```
nltk.download('all')
```

✓ If all the packages are not needed, then individual packages can also be installed by passing its name in nltk.download().

Syntax : `nltk.download('package_name')`

For example : `nltk.download('punkt')`

Pre-processing steps for text analytics using NLTK :

➢ **Tokenization :** It is the first step to perform text analytics. Tokenization means breaking down a textual paragraph into small chunks such as words or sentences. It is classified into two sections :

✓ **Sentence Tokenization and Word Tokenization :** Sentence Tokenization breaks the text into sentences whereas Word Tokenization breaks the text into words.

Example :

```
# Import sent_tokenize and word_tokenize package belonging to nltk
from nltk.tokenize import sent_tokenize
from nltk.tokenize import word_tokenize

# Take some textual content to tokenize it in sentences and words.
paragraph_text="""Hello all, Welcome to Python Programming Academy. Python
Programming Academy is a nice platform to learn new programming skills. It is
difficult to get enrolled in this Academy."""

# Supply textual content to sent_tokenize() and word_tokenize()
tokenized_text_data=sent_tokenize(paragraph_text)
tokenized_words=word_tokenize(paragraph_text)
print("Tokenized Sentences : \n", tokenized_text_data, "\n")
print("Tokenized Words : \n",tokenized_words, "\n")
```

Output :

```
Tokenized Sentences :
 ['Hello all, Welcome to Python Programming Academy.', 'Python Programming Acade
my is a nice platform to learn new programming skills.', 'It is difficult to get
enrolled in this Academy.']

Tokenized Words :
   ['Hello', 'all', ',', 'Welcome', 'to', 'Python', 'Programming', 'Academy', '.',
  'Python', 'Programming', 'Academy', 'is', 'a', 'nice', 'platform', 'to', 'learn'
, 'new', 'programming', 'skills', '.', 'It', 'is', 'difficult', 'to', 'get', 'en
rolled', 'in', 'this', 'Academy', '.']
```

➢ **Frequency Distribution :**

✓ The frequency distribution helps to understand how many words have occurred how many times in the given textual data.

Example :

```
# Import word_tokenize
from nltk.tokenize import word_tokenize
# Import FreqDist package belonging to nltk.probability
from nltk.probability import FreqDist
# Textual data for word tokenization
```

```
paragraph_text="""Hello all, Welcome to Python Programming Academy. Python
Programming Academy is a nice platform to learn new programming skills. It is
difficult to get enrolled in this Academy."""
# Word Tokenization
tokenized_words=word_tokenize(paragraph_text)
# Pass the tokenized words to FreqDist
frequency_distribution=FreqDist(tokenized_words)
print(frequency_distribution)
```

Output :

```
<FreqDist with 24 samples and 32 outcomes>
```

✓ To find most common words using Frequency Distribution, add the following lines in above code :

```
print(frequency_distribution.most_common(2))
```

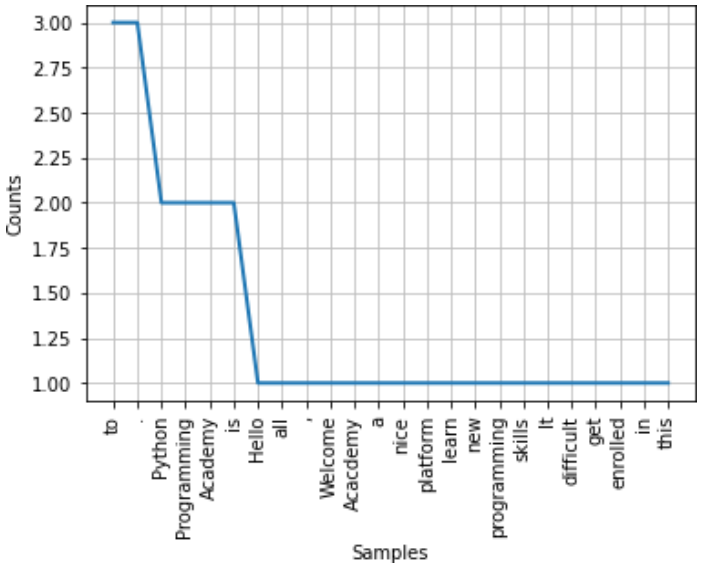
Output :

```
[('to', 3), ('.', 3)]
```

✓ To plot the Frequency Distribution, add the following lines of code :

```
import matplotlib.pyplot as plt
frequency_distribution.plot(32,cumulative=False)
plt.show()
```

Output :



➢ **Stopwords :** Stopwords are considered as Noise in textual data. For example if text is containing words such as is, are, am, a, this, the, an etc. then they are treated as stopwords.

✓ These stopwords needs to be removed from actual text for further processing. Using NLTK, first identify and create a list of stopwords in given text. Then remove it from the original content. Before working with stopwords, make sure to download it by using following :

```
import nltk
nltk.download('stopwords')
```

To check list all Stopwords :

```
from nltk.corpus import stopwords
# It will find the stowords in English language.
stop_words_data=set(stopwords.words("english"))
print(stop_words_data)
```

Output :

```
{'wouldn', 'down', 'was', 'any', 'themselves', 'on', 'how', 'y', 'them', 'do',
'as', "couldn't", 'wasn', 'can', 'yourself', "mightn't", 'm', "wasn't", 'yours',
"haven't", 'have', 'their', 'from', 'with', 'through', 'been', 'couldn', 'here',
'your', 'above', 'same', 'ours', 'now', 'isn', 'that', 'just', 'further',
'only', "won't", 'having', 'these', 'won', 'himself', 'ourselves', 'which',
"you're", 'while', 'of', "doesn't", "should've", "mustn't", 'hadn', 'are',
'not', 'he', 'she', 'am', 'an', 'most', 'whom', 'where', 'than', 'didn',
"isn't", 'shouldn', 'what', 'mustn', 'some', 'very', 'should', 'ain', "you'd",
'yourselves', 'own', 'but', 'we', 't', 'out', 'such', 'in', 've', 'this',
'shan', 'about', 'over', 'both', 'all', 'why', 'i', 'being', "wouldn't", 'll',
'myself', 'between', 'has', "didn't", 'hers', 'hasn', "she's", 'other', 'if',
'itself', 'below', "aren't", 'too', 'under', 'herself', 'be', 'after', 'off',
're', 'during', 'until', 'our', "shouldn't", 'into', 'don', 'again', 'nor',
'needn', "that'll", "weren't", 'no', 'so', 'then', 'before', 'his', 'its',
'few', 'doing', "don't", "you'll", "hadn't", 'because', 'there', 'did', 'my',
"needn't", "it's", 'they', 'for', 'does', 'is', 'a', 'against', 'who', 'and',
"shan't", 'o', 'weren', 'him', 'or', 'theirs', 'were', 'had', 'doesn', 'you',
'haven', 'those', 'me', 'when', 's', 'd', 'it', 'up', 'by', 'each', 'once',
'aren', "you've", 'her', "hasn't", 'to', 'more', 'will', 'mightn', 'the', 'at',
'ma'}
```

Removing Stopwords :

The above words in the output are predefined stopwords in English Language. If either of these words occur in a user-defined textual data, then it can be removed as follows :

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
# Textual data to remove stopwords
paragraph_text="""Hello all, Welcome to Python Programming Academy. Python
Programming Academy is a nice platform to learn new programming skills. It is
difficult to get enrolled in this Academy."""
# Word Tokenization
tokenized_words=word_tokenize(paragraph_text)
# It will find the stowords in English language.
stop_words_data=set(stopwords.words("english"))
# Create a stopwords list to filter it from original text
filtered_words_list=[]
for words in tokenized_words:
    if words not in stop_words_data:
        filtered_words_list.append(words)
print("Tokenized Words : \n",tokenized_words,"\n")
print("Filtered Words : \n",filtered_words_list,"\n")
```

Output :

```
Tokenized Words :

 ['Hello', 'all', ',', 'Welcome', 'to', 'Python', 'Programming', 'Academy', '.',
'Python', 'Programming', 'Academy', 'is', 'a', 'nice', 'platform', 'to',
'learn', 'new', 'programming', 'skills', '.', 'It', 'is', 'difficult', 'to',
'get', 'enrolled', 'in', 'this', 'Academy', '.']

Filtered Words :

 ['Hello', ',', 'Welcome', 'Python', 'Programming', 'Academy', '.', 'Python',
'Programming', 'Academy', 'nice', 'platform', 'learn', 'new', 'programming',
'skills', '.', 'It', 'difficult', 'get', 'enrolled', 'Academy', '.']
```

➢ **Stemming :** Stemming is a process of linguistics normalization to reduce words to their word root or divide the derivational affixes. For example : writing, wrote, written can stemmed or reduced as write.

Example :

```
# Same code as previous example to remove stop words from tokenized words
from nltk.stem import PorterStemmer
porter_stemmer=PorterStemmer()
stemmed_text_words=[]
for words in filtered_words_list:
    stemmed_text_words.append(porter_stemmer.stem(words))
print("Filtered Words : \n",tokenized_words,"\n")
print("Stemmed Words : \n",stemmed_text_words,"\n")
```

➢ **Lemmatization :** Lemmatization is a process of removing words to their base words which is linguistically correct lemmas. For example : “Running” word will be lemmatized to “run”. Before that download the package “wordnet” belonging to nltk as follows :

```
import nltk
nltk.download('wordnet')
# Lemmatization
from nltk.stem.wordnet import WordNetLemmatizer
lemmatizer=WordNetLemmatizer()
word_text="running"
print("Lemmatized Word : ",lemmatizer.lemmatize(word_text,"v"))
```

Output :

```
Lemmatized Word :  run
```

➢ **POS Tagging :** The POS (Part-of-Speech) tagging is basically used to identify the grammatical group of given words i.e. Noun, Pronoun, Verb, Adjective, Adverbs etc. on the basis of its context.
Before that download the package “averaged_perceptron_tagger” belonging to nltk as follows :

```
import nltk
nltk.download('averaged_perceptron_tagger')
# Part-of-Speech Tagging
import nltk
```

```
from nltk.tokenize import word_tokenize
text_data="Hello all, Welcome to Python programming"
tokenized_data=word_tokenize(text_data)
print(nltk.pos_tag(tokenized_data))
```

Output :

```
[('Hello', 'NNP'), ('all', 'DT'), (',', ','), ('Welcome', 'NNP'), ('to', 'TO'),
('Python', 'NNP'), ('programming', 'NN')]
```

Text Summarization :

Text summarization is an NLP technique that extracts text from a large amount of data. It is the process of identifying the most important meaningful information in a document and compressing it into a shorter version by preserving its meaning. Types: Extractive summarization and Abstractive summarization

To perform extractive summarization, we calculate the sentence weights and choose the first 'n' sentences with maximum weight. The weights are calculated on the basis of the word frequencies

Steps:

1. Preprocess the text
2. Create the word frequency table
3. Tokenize the sentence
4. Score the sentences: Term frequency
5. Generate the summary

Sample code

```
import nltk
nltk.download('all')
#Preprocessing
import re
text="""

 Large paragraph of text
"""
text = re.sub(r'[[0-9]*]', ' ', text)
text = re.sub(r's+', ' ', text)
# Removing Square Brackets, digits, special symbols
import re
text = re.sub(r'[[0-9]{}*]', ' ', text)
# Removing special characters and digits
formatted_text = re.sub('[^a-zA-Z]', ' ', text)
#Here the formatted_article_text contains the formatted article.
#We will use this object to calculate the weighted frequencies and we will replace the weighted
#frequencies with words in the text object.
#Calculate the frequency of occurrence of each word. To find the weighted frequency, we divide the
#number of occurrences of all the words by the frequency of the most occurring word.
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stopWords = set(stopwords.words("english"))
words = word_tokenize(formatted_text)
```

```
# Creating a frequency table of words
wordfreq = {}
for word in words:
    if word in stopWords:
        continue
    if word in wordfreq:
        wordfreq[word] += 1
    else:
        wordfreq[word] = 1
#Compute the weighted frequencies
maximum_frequency = max(wordfreq.values())
for word in wordfreq.keys():
    wordfreq[word] = (wordfreq[word]/maximum_frequency)
# Creating a dictionary to keep the score # of each sentence
sentences = sent_tokenize(text)
sentenceValue = {}
for sentence in sentences:
    for word, freq in wordfreq.items():
        if word in sentence.lower():
            if sentence in sentenceValue:
                sentenceValue[sentence] += freq
            else:
                sentenceValue[sentence] = freq
import heapq
summary = ''
summary_sentences = heapq.nlargest(4, sentenceValue, key=sentenceValue.get)
summary = ' '.join(summary_sentences)
print(summary)
```

Sentiment Analysis using NLTK :

Sentiment analysis is a technique which detects the underlying sentiment on specific textual content. It is considered as a process of classifying text on the basis of labels i.e. positive, negative or neutral. To perform Sentiment Analysis using NLTK, it has a supportable package named as VADER. VADER stands for **Valence Aware Dictionary and Sentiment Reasoner**. It is a lexicon and rule-based sentiment analysis tool which is specifically used to identify the expressed sentiments. It is beneficial to use VADER on social media data since it not only gives the analysis as whether the text is positive or negative, but it also tells about the intensity of text i.e. how much positive or negative the text is. To use VADER, first download "vader_lexicon" package belonging to nltk.

```
import nltk
nltk.download('vader_lexicon')
```

Examples : Let's consider some text statements expressing different emotions and analyzing them using VADER.

Example 1 :

```
# Import SentimentIntensityAnalyzer from vader package
from nltk.sentiment.vader import SentimentIntensityAnalyzer
# Create an object of SentimentIntensityAnalyzer()
```

```
vader_analyzer=SentimentIntensityAnalyzer()
text1="I am feeling good" # The text is positive.
print(vader_analyzer.polarity_scores(text1))
```

Output :

```
{'neg': 0.0, 'neu': 0.185, 'pos': 0.815, 'compound': 0.5267}
```

Here the output is giving some labels :

- neg : Negative
- neu : Neutral
- pos : Positive

It has given 'pos' value as 0.815 which is maximum of all the other values since the statement is positive. Similarly, we can check it on other emotions as well.

Example 2 :

```
text1="I hate tea"
print(vader_analyzer.polarity_scores(text1))
```

Output :

```
{'neg': 0.787, 'neu': 0.213, 'pos': 0.0, 'compound': -0.5719}
```

Example 3 : Consider the following example to get the overall rating about a statement i.e. overall whether it is positive, negative or neutral.

```
text1="I like Python"
result1=vader_analyzer.polarity_scores(text1)

# To find percentage of ratings
print("The sentence is rated as ",result1['pos']*100,"% Positive")
print("The sentence is rated as ",result1['neg']*100,"% Negative")
print("The sentence is rated as ",result1['neu']*100,"% Neutral")
if result1['compound']>=0.05:
    print("Overall rating for sentence is Positive")
elif result1['compound']<=-0.05:
    print("Overall rating for sentence is Negative")
else:
    print("Overall rating for sentence is neutral")
```

```
Output :

The sentence is rated as  71.39 % Positive
The sentence is rated as 0.0 % Negative
The sentence is rated as 28.59 % Neutral
Overall rating for sentence is Positive
```

Word Cloud for Sentiment Analysis :

Word cloud is basically a data visualization technique to represent the textual content where the size of each visualized word implies its importance, frequency and intensity. It is a good tool to visualize the text and perform sentiment analysis to find the frequency of words having positive, negative or neutral emotions.

✓ To install wordcloud, use the following command :

```
pip install wordcloud
```

✓ **Example :** Download the movie_review.csv dataset from Kaggle by using the following link : https://www.kaggle.com/nltkdata/movie-review/version/3?select=movie_review.csv

Sample Data from movie_review.csv

fold_id	cv_tag	html_id	sent_id	text	tag
0	cv000	29590	0	films adapted from comic books have had plenty of suc	pos
0	cv000	29590	1	for starters , it was created by alan moore (and eddie c	pos
0	cv000	29590	2	to say moore and campbell thoroughly researched the s	pos
0	cv000	29590	3	the book (or " graphic novel , " if you will) is over 500 p	pos
0	cv000	29590	4	in other words , don't dismiss this film because of its so	pos
0	cv000	29590	5	if you can get past the whole comic book thing , you mi	pos
0	cv000	29590	6	getting the hughes brothers to direct this seems almost	pos
0	cv000	29590	7	the ghetto in question is , of course , whitechapel in 18	pos
0	cv000	29590	8	it's a filthy , sooty place where the whores (called " unf	pos
0	cv000	29590	9	when the first stiff turns up , copper peter godley (robl	pos
0	cv000	29590	10	abberline , a widower , has prophetic dreams he unsucc	pos
0	cv000	29590	11	upon arriving in whitechapel , he befriends an unfortun	pos
0	cv000	29590	12	i don't think anyone needs to be briefed on jack the ripp	pos
0	cv000	29590	13	in the comic , they don't bother cloaking the identity of	pos
0	cv000	29590	14	it's funny to watch the locals blindly point the finger of	pos

Now to perform sentiment analysis on above dataset and creating a wordcloud, consider the following code : (Here, we will represent Positive words with green color, Negative words with red color and Neutral words with white color)

```
# Import the necessary packages
from nltk.corpus import stopwords
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from wordcloud import WordCloud, get_single_color_func
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Read Movies Reviews Data CSV file.
movies_data=pd.read_csv('movie_review.csv')
# Convert the column data type from ndarray to string.
movies_reviews=movies_data['text'].values.astype(str)
movies_reviews1=np.array_str(movies_reviews)
# It will find the stowords in English language.
stop_words_data=set(stopwords.words("english"))
words=movies_reviews1.split()

# Remove Duplicate Values
final_data=[]
for w in words:
    if w not in final_data:
        final_data.append(w)
```

```
# Create dictionaries to store positive and negative words with polarity.
positive_words=dict()
negative_words=dict()

# Create lists to store positive and negative words without polarity.
positive=[]
negative=[]

# Sentiment Analysis
sentiment_analyzer=SentimentIntensityAnalyzer()
for i in words:
    if not i.lower() in stop_words_data: # It will remove stopwords.
        polarity=sentiment_analyzer.polarity_scores(i)
        if polarity['compound']>=0.05: # Positive Sentiment
            positive_words[i]=polarity['compound']
        if polarity['compound']<=-0.05: # Negative Sentiment
            negative_words[i]=polarity['compound']

# Append the positive and negative words from dictionaries to lists i.e.
positive[] and negative[]
for key,value in positive_words.items():
    positive.append(key)
for key,value in negative_words.items():
    negative.append(key)

# Create a dictionary to mention the colors : green for positive and red for
negative
coloured_words={"green":positive,"red":negative}

# Implement separate colour assignments
class ColourAssignment(object):
    # Functions to give different colours on the basis of sentiments.
    def _init_(self,coloured_words,default):
        self.coloured_words=[
            (get_single_color_func(colour),set(words))
            for (colour,words) in coloured_words.items()]
        self.default=get_single_color_func(default)

    def get_colour(self,word):
        try:
            colour=next(
                colour for (colour,words) in self.coloured_words
                if word in words)
        except StopIteration:
            colour=self.default
        return colour

    def __call__(self,word, **kwargs):
        return self.get_colour(word) (word, **kwargs)
```

```
# To print the plot
word_cloud=WordCloud(collocations=False,background_color='black').generate(movie
s_reviews1)
# Neutral words will be visible as black
group_color=ColourAssignment(coloured_words, 'white')
word_cloud.recolor(color_func=group_color)
plt.figure()
plt.imshow(word_cloud, interpolation="bilinear")
plt.axis("off")
plt.show()
```

Output :

Social Media Data Analytics :

When it comes to social media such as Twitter, Facebook, YouTube etc., bulk of data is available which is needed to be examined or analyzed for interpreting the opinions of people conveyed in different formats. It basically puts the subjective information in the form emotions.

Twitter Data Analysis :

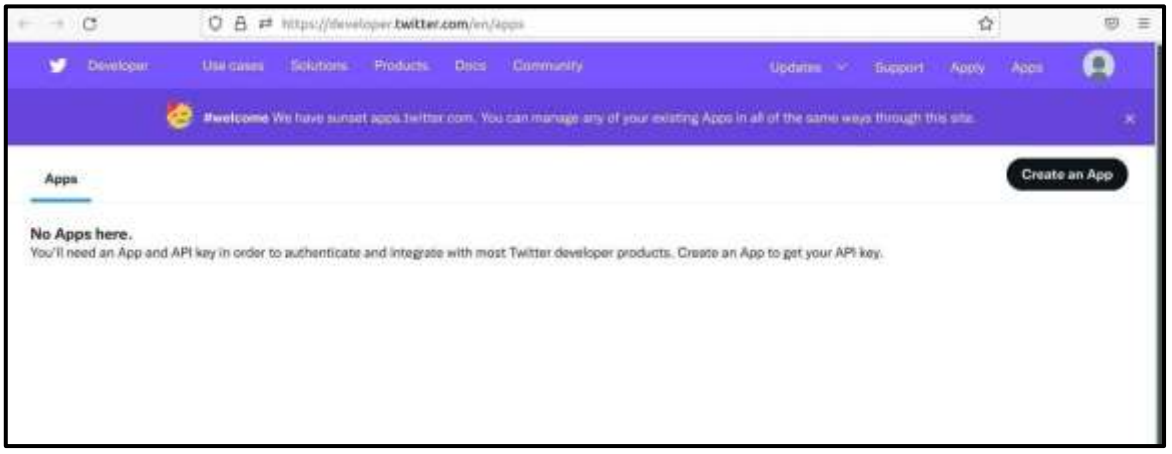
✓ To perform the analysis on twitter data, first we need to get the data.
✓ There are multiple ways to get Twitter data. Some are :
- Getting tweets through twitter API and analyzing them.
- Downloading the Twitter datasets online (Example : Kaggle)

Getting tweets through twitter API and analyzing the tweets :

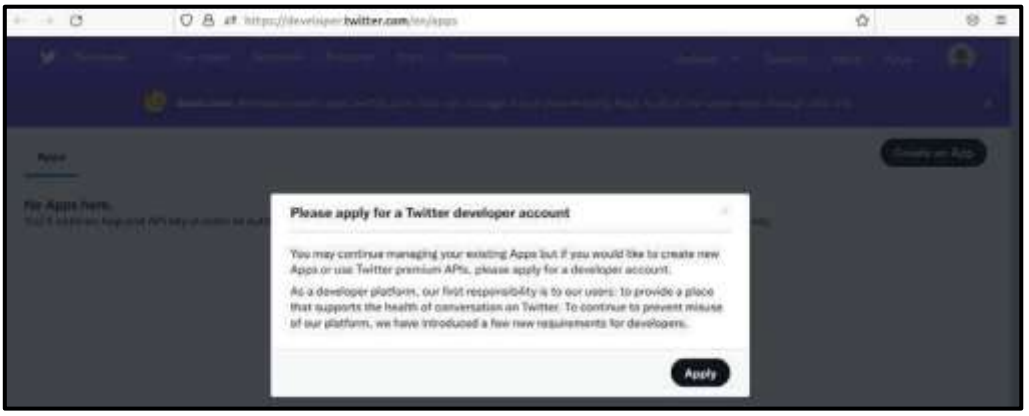
To get the tweets through twitter API, Twitter account is needed and App is to be registered. Follow the below steps :

❖ First create a Twitter account if you do not have one. Visit to https://twitter.com/i/flow/signup and create an account. Existing account can also be used.

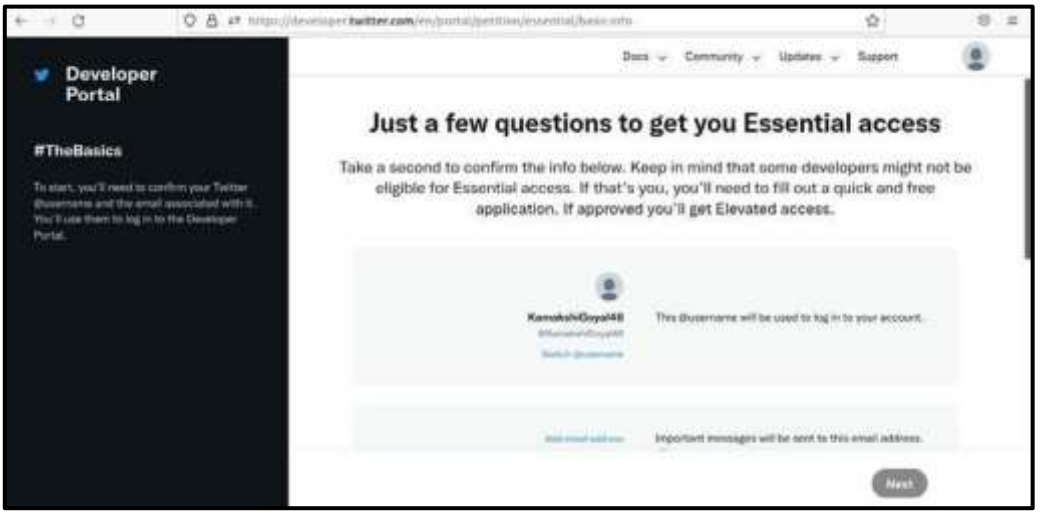
❖ Now create an App on Twitter Developer using following link : https://developer.twitter.com/en/apps

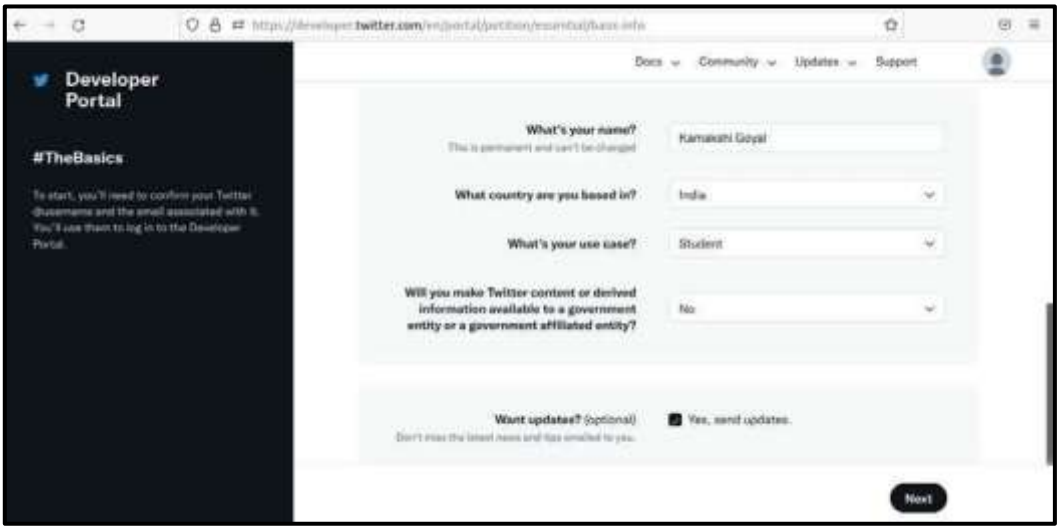


❖ Now click on "Create an App" button to create an application to get the API key for credentials. It will ask to apply for a Developer Account.



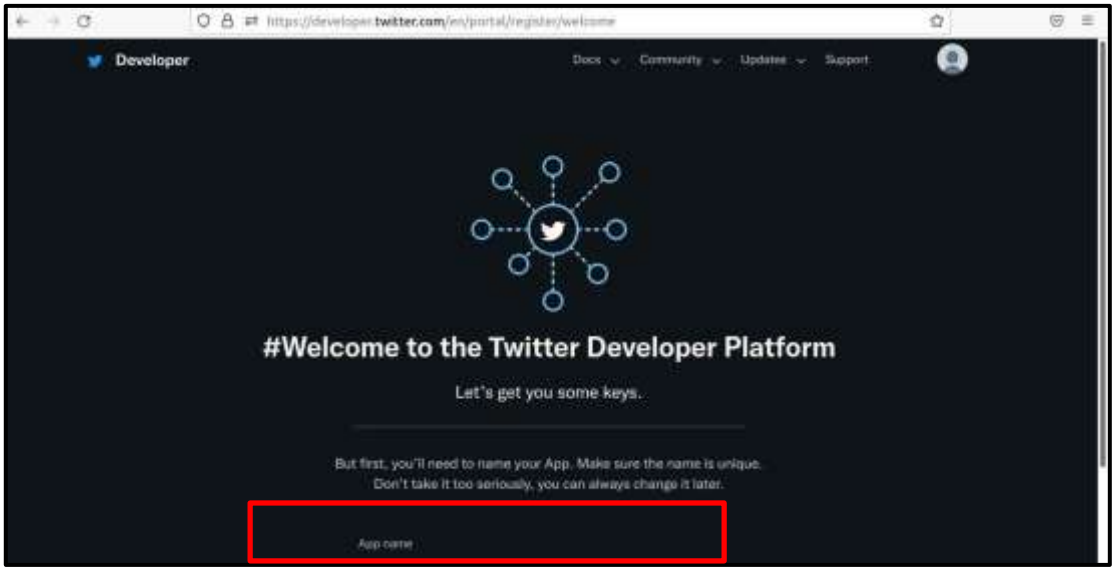
❖ Click on Apply and continue. And then answer the questions visible on the screen.



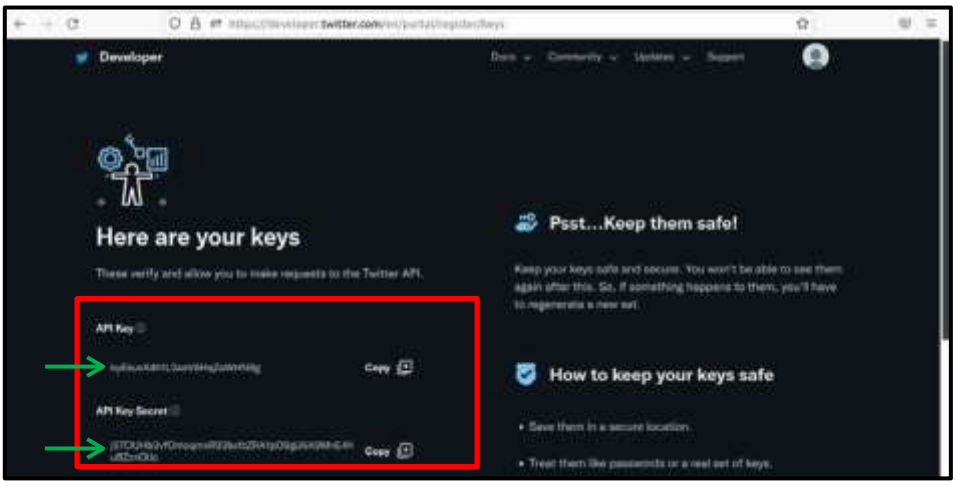


❖ After submitting the request, you will receive a message from twitter to get the Email confirmation. Then we can get the keys. Visit the following link for App creation.

https://developer.twitter.com/en/portal/register/welcome



❖ Now give the App name and click on Get Keys.



❖ **Libraries used for twitter data analysis :**

1. **tweepy :** It is a Python library which is used to access the Twitter API. To install tweepy, use the following command :

```
pip install tweepy
```

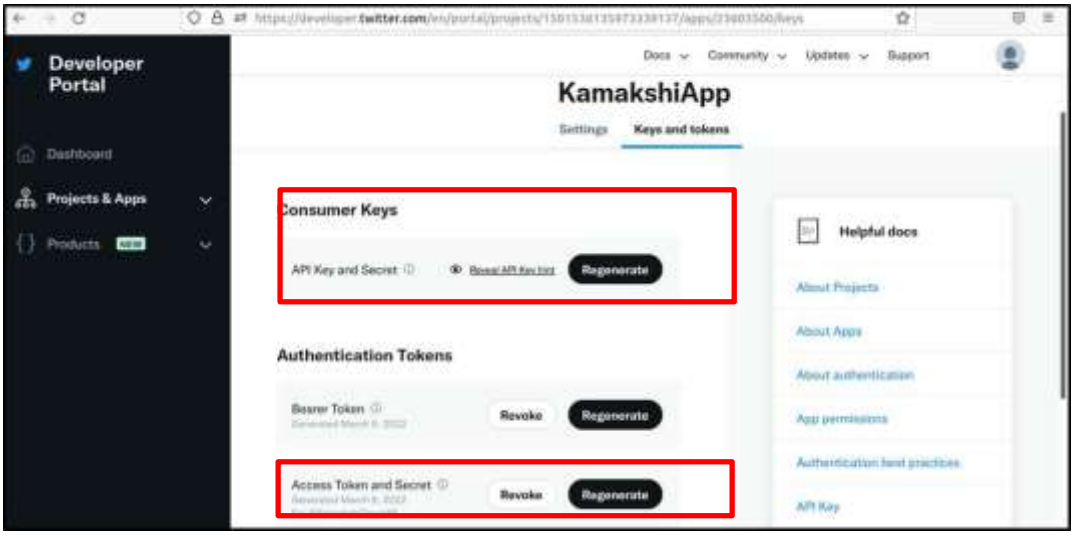
Import the necessary library as :

```
import tweepy
```

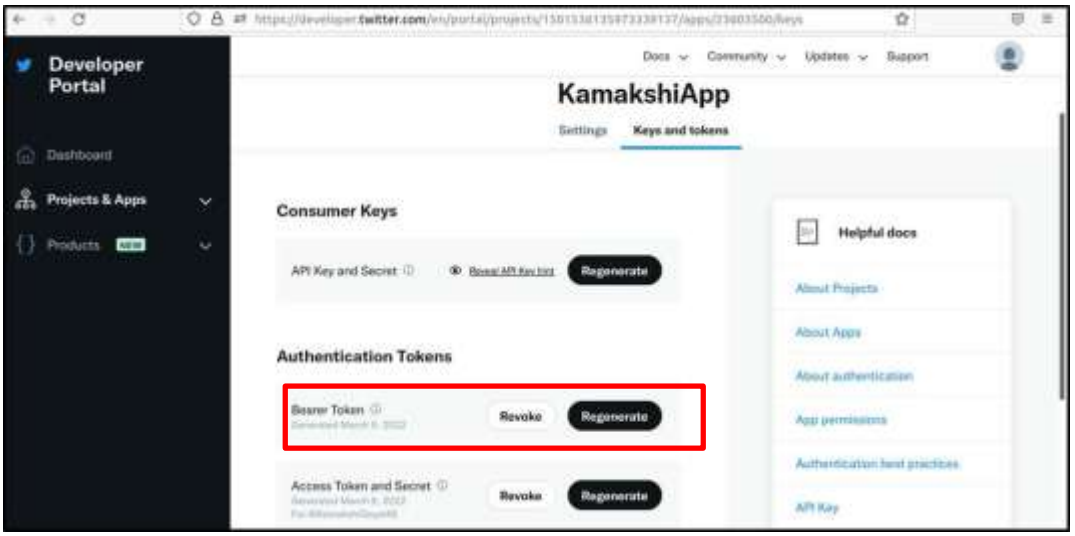
❖ **Twitter API Authentication :**

To perform Twitter API authentication, we have multiple options :
We can perform authentication using :

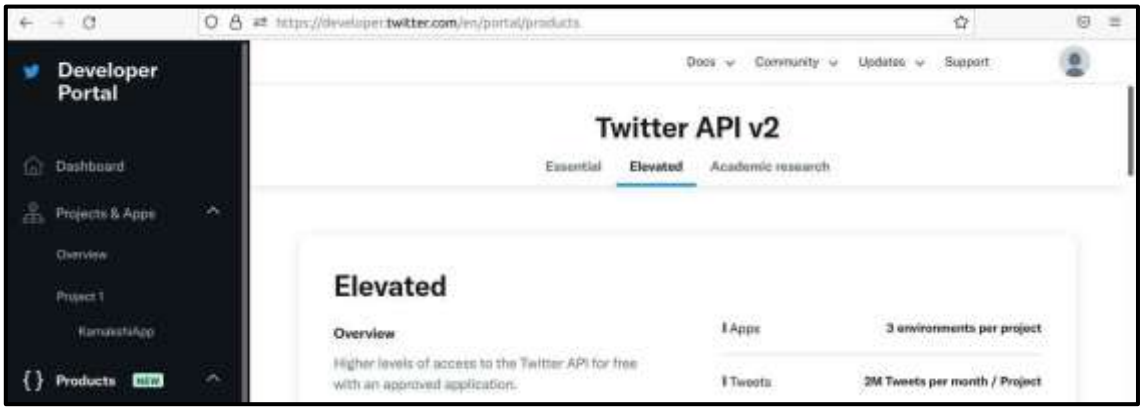
- Consumer Key
- Consumer Secret Key
- Access Token
- Access Token Secret

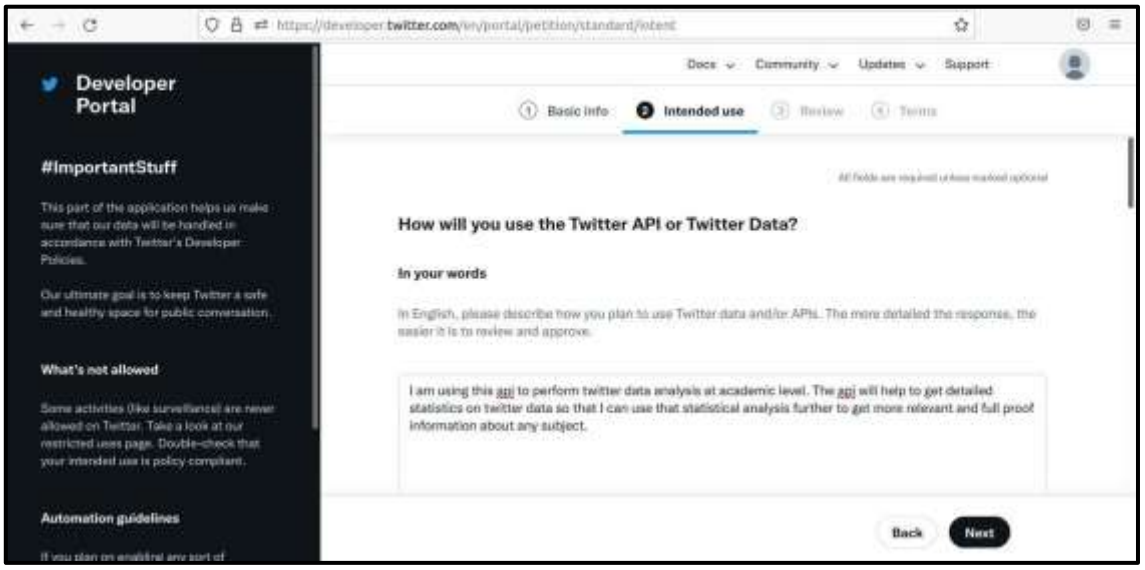


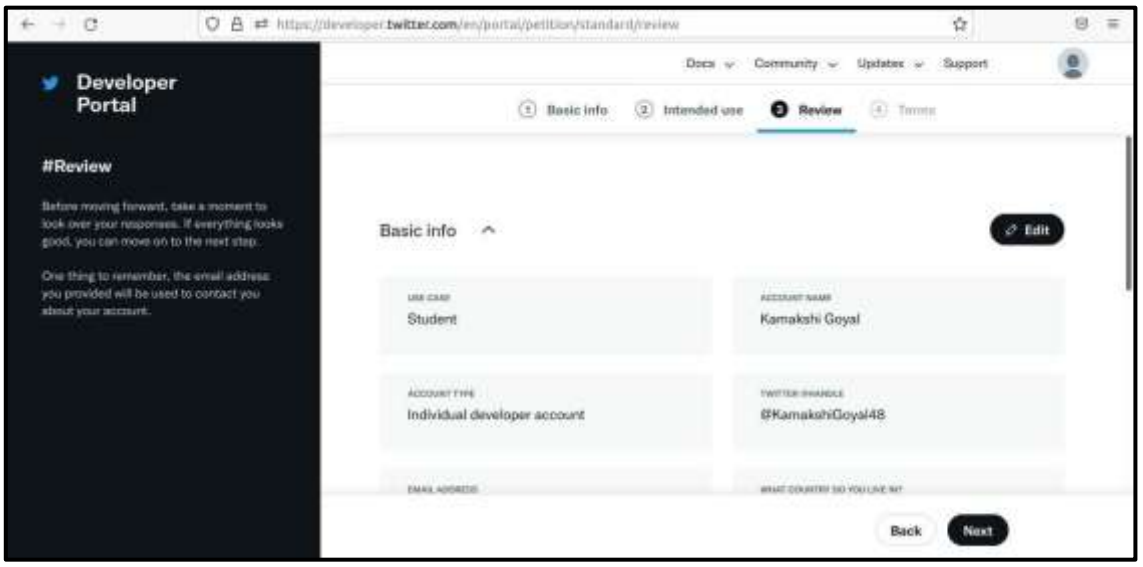
Or you can also use the "Bearer Token" to perform authentication. For this code, Bearer Token is used. If you want to use another approach, refer this : https://docs.tweepy.org/en/stable/authentication.html. You can find Bearer Token here :

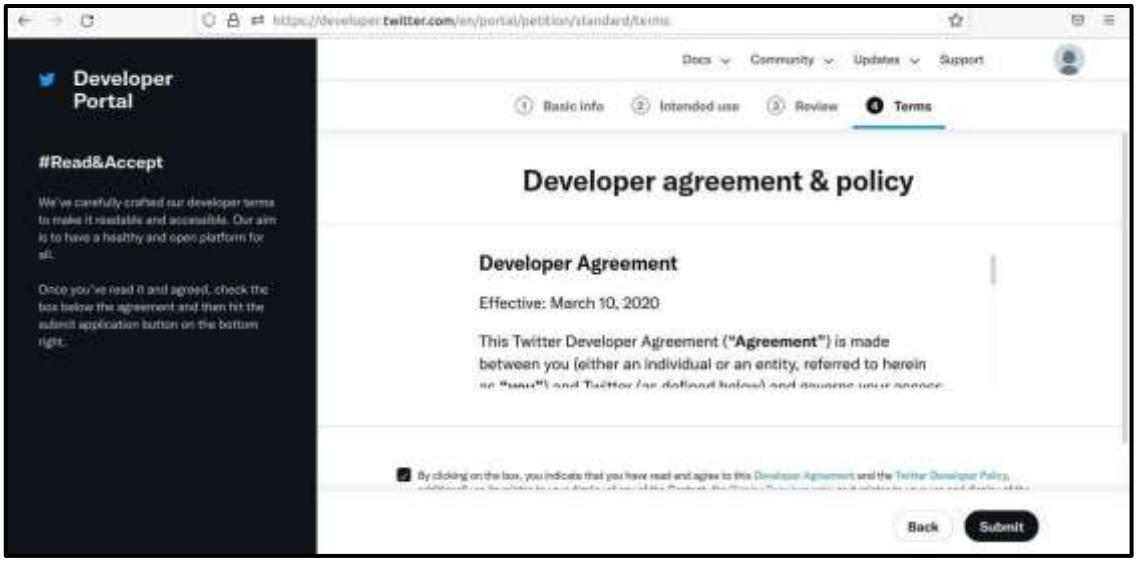


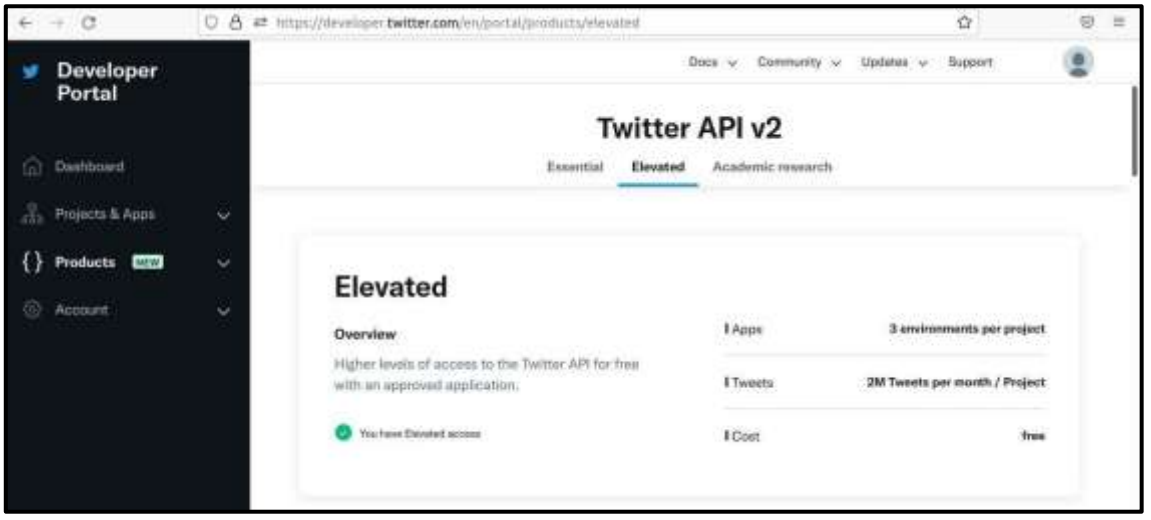
Before using Bearer Token, make sure to use “Elevated” section of App. When first time app gets created, it comes with “Essential”. But to use Bearer Token directly, “Elevated” is to be used.











In case if the token gets expired, then it can be regenerated as well. Add the following lines of code :

```
auth=tweepy.OAuth2BearerHandler("Your Bearer Token")
api=tweepy.API(auth)
```

❖ Getting tweets having Hash Tags or by using Keywords :

```
# Get the tweets on the basis of Hash Tags or Keywords.
search_tag=input("Enter the Hash Tag or Keyword for which you want to get the tweets : ")
```

```
no_of_tweets=int(input("How many tweets you want ? "))
# Iterate over the tweets.
tweets=tweepy.Cursor(api.search_tweets, q=search_tag).items(no_of_tweets)
# Create a list to store all the tweets.
tweet_list=[]
for tweet in tweets:
    tweet_list.append(tweet.text)

print(tweet_list)
```

Output : (Example)

```
Enter the Hash Tag or Keyword for which you want to get the tweets : #sadhguru

How many tweets you want ? 5

['RT @gauravsingh_ss: 2- @SadhguruJV &amp;amp; @cpsavesoil now "ST. XAVIER\'S HIGH SCHOOL" (Chapra, Bihar) have also taken responsibility of #SaveSoil…', 'RT @TUndercoverMonk: Get ready for the BIGGEST Environment movement on the planet. \nThe intention is to make #Soil as the MAIN talking poin…', 'RT @SouvikMitra94: @SadhguruJV @iviviannrichards @BeefyBotham @cpsavesoil #Sadhguru and #SirVivRichards in same frame to #SaveSoil □□ https:…', "RT @PenguinIndia: Read this #exclusive excerpt from @MansinghVivek's latest release, where @SadhguruJV talks about the role of the mind, bo…", 'RT @AntiguaOpm: #InPhotos Today, PM @gastonbrowne met with @SadhguruJV and @machelmontano ahead of the launch of a global ecological campai…']
```

❖ **More Analysis on Twitter Data :** We can further perform different analysis on gathered data as follows :

- ✓ First select the user ID on which analysis is to be done.
- ✓ Then we can find various information related to tweets such as 'created_at', 'id', 'id_str', 'text', 'truncated', 'entities', 'metadata', 'source', 'in_reply_to_status_id', 'in_reply_to_status_id_str', 'in_reply_to_user_id', 'in_reply_to_user_id_str', 'in_reply_to_screen_name', 'user', 'geo', 'coordinates', 'place', 'contributors', 'retweeted_status', 'is_quote_status', 'retweet_count', 'favorite_count', 'favorited', 'retweeted', 'lang', 'possibly_sensitive'.

Example :

```
# Select a specific user by using a twitter user ID.
user_id=input("Enter a Twitter user ID : ")
no_of_tweets=int(input("How many tweets you want ? "))

# To get tweets details such as tweet ID,
tweets=api.user_timeline(screen_name=user_id, count=no_of_tweets, include_rts=False, tweet_mode="extended")
for tweet_info in tweets:
    print("Tweet ID : ",tweet_info.id) # Tweet ID
    print("Created at : ",tweet_info.created_at) # Date on which tweet is created.
    print("Tweet : ",tweet_info.full_text) # Tweet
    print("Retweet count : ",tweet_info.retweet_count) # Number of retweets on each tweet.
```

```
    print("\n")
```

Output : (Example)

```
Enter a Twitter user ID : SadhguruJV

How many tweets you want ? 5
Tweet ID :  1502151438419464196
Created at :  2022-03-11 05:16:26+00:00
Tweet :  Sir Vivian Richards &amp;amp; Lord Ian Botham - a joy to meet you during my
Antigua visit for the #SaveSoil movement. Your achievements in cricket &amp;amp;
beyond are commendable. Please join me in restoring our world's Soil, the basis
of all Life on Earth. -Sg @iviviannrichards @BeefyBotham https://t.co/M53Ckhu0Lg
Retweet count : 1736


Tweet ID :  1502113329103487012
Created at :  2022-03-11 02:45:00+00:00
Tweet :  Kriya Yoga requires nothing but dedication towards the practice. As you
refine your energies, there is no way you can remain untransformed.
#SadhguruQuotes https://t.co/byjrSIld2u
Retweet count :  1696


Tweet ID :  1501771371562471426
Created at :  2022-03-10 04:06:11+00:00
Tweet : Congratulations @CISFHQrs for your courageous &amp;amp; committed
contribution to Nation Building for more than five decades. Bharat is proud
&amp;amp; grateful for your stellar service. May you continue to inspire Peace &amp;amp;
Prosperity. Best Wishes. -Sg #CISFRaisingDay2022
Retweet count :  1595


Tweet ID :  1501750941187551232
Created at :  2022-03-10 02:45:00+00:00
Tweet :  You cannot change the past. You can only experience the present moment.
The future must be crafted the way you want. #SadhguruQuotes
https://t.co/eTCAmU3gOl
Retweet count :  2510


Tweet ID :  1501624364889825281
Created at :  2022-03-09 18:22:02+00:00
Tweet :  Machel, #VelliangiriMountains are a Cascade of Grace. Their Power has
empowered millions &amp;amp; will continue to empower future populations. Wonderful
your #Sadhanapada culminated here; it was beautiful to have you &amp;amp; Renee.
Journey on- sing, dance, also transform lives. Blessings. -Sg
https://t.co/y2qV6EBM2k
Retweet count :  1483
```

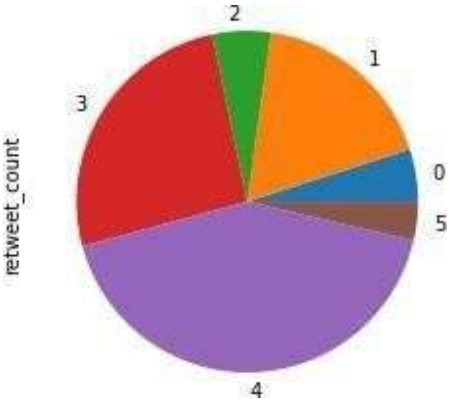
❖ **Visualizing Twitter Data :** We can visualize the twitter data in multiple ways on the basis of attributes returned by Twitter API.

Example : To visualize the number of re-tweets on each tweet :
First create a DataFrame so that it will become easy to get the attributes of Twitter API.
Then create a plot (e.g. Pie Plot) to get the number of re-tweets on each tweet.

```
import tweepy
import pandas as pd
import matplotlib.pyplot as plt

auth=tweepy.OAuth2BearerHandler("Your Bearer Token")
api=tweepy.API(auth)

# Select a specific user by using a twitter user ID.
user_id=input("Enter a Twitter user ID : ")
no_of_tweets=int(input("How many tweets you want ? "))

# To get tweets details such as tweet ID,
tweets=api.user_timeline(screen_name=user_id, count=no_of_tweets, include_rts=False,
tweet_mode="extended")

# Take lists to store different data to create a DataFrame.
tweet_id=[]
tweet_created_at=[]
tweet_full_text=[]
tweet_retweet_count=[]
tweet_favorite_count=[]

for tweet_info in tweets:
    tweet_id.append(tweet_info.id)
    tweet_created_at.append(tweet_info.created_at)
    tweet_full_text.append(tweet_info.full_text)
    tweet_retweet_count.append(tweet_info.retweet_count)
    tweet_favorite_count.append(tweet_info.favorite_count)

twitter_data={'id':tweet_id,'created_at':tweet_created_at,'full_text':tweet_full_text,'r
etweet_count':tweet_retweet_count,'favorite_count':tweet_favorite_count}
# DataFrame
twitter_dataframe=pd.DataFrame(twitter_data)
# Plotting Pie Graph for retweets on each tweet.
twitter_dataframe['retweet_count'].plot.pie()
plt.show()
```

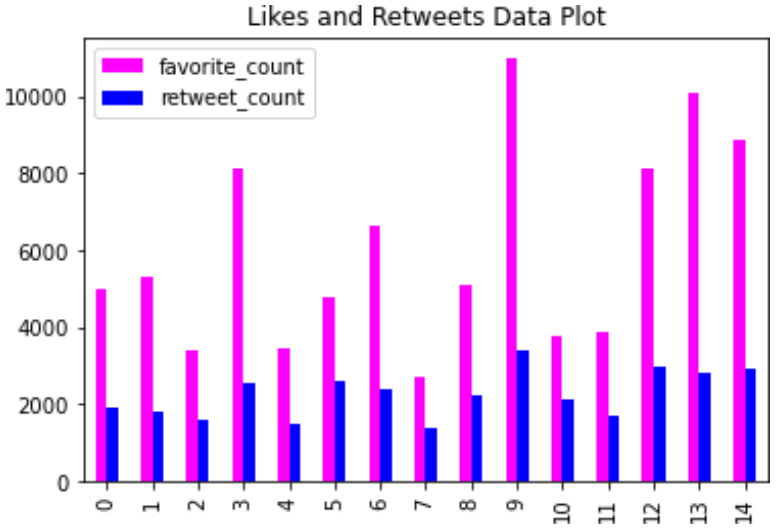
Output :

```
Enter a Twitter user ID : Tesla
How many tweets you want ? 10
```



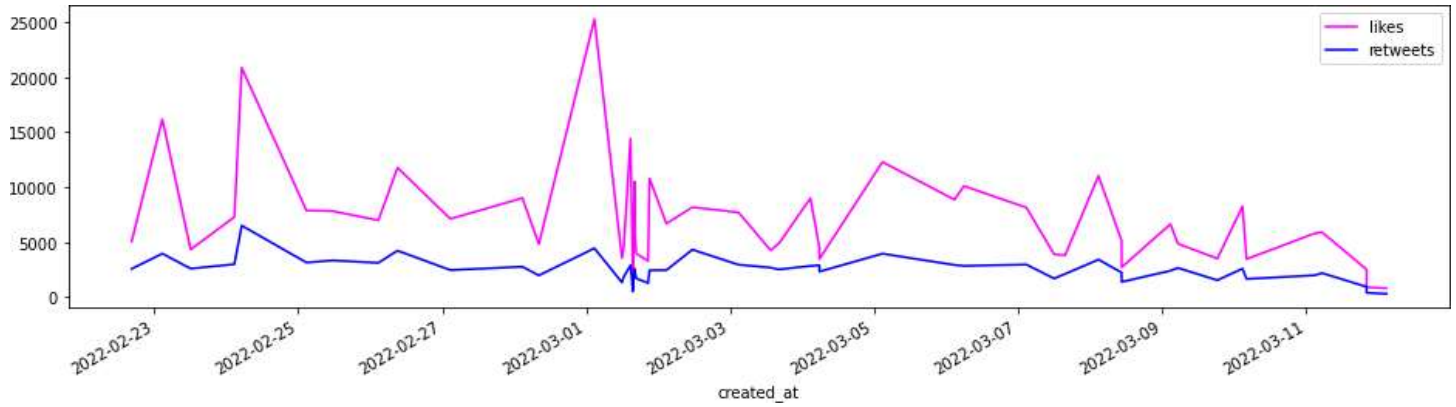
Now to plot the likes and re-tweets received on each tweet, add the following script :

```
# Plot for likes and retweets.
twitter_dataframe.plot.bar(y=['favorite_count','retweet_count'],color=['magenta','blue']
)
plt.show()
```



Consider the following script to plot the Time Series for likes and re-tweets along with dates on which the tweets were published.

```
# Time Series
time_likes=pd.Series(data=twitter_dataframe['favorite_count'].values,index=twitter_dataf
rame['created_at'])
time_likes.plot(figsize=(16,4),label="likes",legend=True,color="magenta")

time_retweets=pd.Series(data=twitter_dataframe['retweet_count'].values,index=twitter_dat
aframe['created_at'])
time_retweets.plot(figsize=(16,4),label="retweets",legend=True,color="blue")

plt.show()
```



✓ You can also get information regarding tweets as total number of likes and re-tweets on each tweet, which tweet has maximum count of likes and got maximum re-tweets.

```
for tweet_info in tweets:
    print("Tweet ID : ",tweet_info.id) # Tweet ID
    print("Created at : ",tweet_info.created_at) # Date on which tweet is created.
    print("Tweet : ",tweet_info.full_text) # Tweet
    print("Retweet count : ",tweet_info.retweet_count) # Number of retweets on each
tweet.
    print("Favorite count : ",tweet_info.favorite_count)
    print("\n")

# To find total number of tweets, likes and retweets on all tweets.
print("Total number of tweets : ",no_of_tweets)
print("Total number of likes on each tweet :
",twitter_dataframe['favorite_count'].sum())
print("Total number of retweets on each tweet :
",twitter_dataframe['retweet_count'].sum())

# To find the number of likes for the most liked tweet.
max_liked_tweet=twitter_dataframe['favorite_count'].max()
print("Number of likes for most liked tweet : ",max_liked_tweet)

# To find the number of retweets for the most retweeted tweet.
max_retweeted_tweet=twitter_dataframe['retweet_count'].max()
print("Number of retweets for the most retweeted tweet : ",max_retweeted_tweet)

# Most liked tweet text.
most_liked_tweet=twitter_dataframe[twitter_dataframe['favorite_count']==twitter_datafram
e['favorite_count'].max()]
print("Most Liked Tweet : ")
print(most_liked_tweet['full_text'])

# Most retweeted tweet text.
most_retweeted_tweet=twitter_dataframe[twitter_dataframe['retweet_count']==twitter_dataf
rame['retweet_count'].max()]
print("Most Retweeted Tweet : ")
print(most_retweeted_tweet['full_text'])
```

Output : (Example)

```
Enter a Twitter user ID : SadhguruJV
How many tweets you want ? 5
Total number of tweets :  5
Total number of likes on each tweet : 16863
Total number of retweets on each tweet :  6180
Number of likes for most liked tweet :  5964
Number of retweets for the most retweeted tweet :  2207

Tweet ID :  1502475716935442442
Created at :  2022-03-12 02:45:00+00:00
Tweet :  Only if you invest your emotions in what matters to you, will life become
powerful and really meaningful. #SadhguruQuotes https://t.co/EWJ2Aneqps
Retweet count : 447
Favorite count :  1247

Tweet ID :  1502375448885432326
Created at :  2022-03-11 20:06:34+00:00
Tweet :  #SaveSoil #MoU #CARICOM
@GastonBrowne @AntiguaOpm @SkerritR @PhilipJPierreLC @pmharriskn @antiguagov
@SaintLuciaGov @skngov @molwynjoseph @SamMarshallMP @machelmontano @armandarton
@GlobalCitizenFo @cpsavesoil @PMOIndia https://t.co/RMXpcgW12d
Retweet count : 461
Favorite count :  1026

Tweet ID :  1502375423451164672
Created at :  2022-03-11 20:06:28+00:00
Tweet :  A historic moment marked by the first #SaveSoil MoUs signed by the pearls of
the ocean. Governments of Antigua &amp;amp; Barbuda, Dominica, St Lucia, and St Kitts &amp;amp;
Nevis — may your commitment to soil revitalization be an inspiration to the rest of the
world. -Sg @CARICOMorg #CARICOM https://t.co/0glWuMlFBy
Retweet count : 1074
Favorite count :  2806

Tweet ID :  1502151438419464196
Created at :  2022-03-11 05:16:26+00:00
Tweet :  Sir Vivian Richards &amp;amp; Lord Ian Botham - a joy to meet you during my Antigua
visit for the #SaveSoil movement. Your achievements in cricket &amp;amp; beyond are
commendable. Please join me in restoring our world’s Soil, the basis of all Life on
Earth. -Sg @ivivianrichards @BeefyBotham https://t.co/M53Ckhu0Lg
Retweet count : 2207
Favorite count :  5964

Tweet ID :  1502113329103487012
Created at :  2022-03-11 02:45:00+00:00
Tweet :  Kriya Yoga requires nothing but dedication towards the practice. As you refine
your energies, there is no way you can remain untransformed. #SadhguruQuotes
https://t.co/byjrSIld2u
Retweet count : 1991
Favorite count :  5820

Most Liked Tweet :
3    Sir Vivian Richards &amp;amp; Lord Ian Botham - a ...
Name: full_text, dtype: object

Most Retweeted Tweet :
3    Sir Vivian Richards &amp;amp; Lord Ian Botham - a ...
Name: full_text, dtype: object
```

❖ **Sentiment Analysis on Twitter Data :** We can also perform sentiment analysis on gathered twitter data. Here two libraries will be needed, i.e. TextBlob and Vader.

1. **textblob :** It is a Python library which is used for processing textual data. It is built on top of NLTK module and offers a simple API to access its methods to perform basic Natural Language Processing tasks. To install textblob, use the following command :
   pip install textblob
2. **VADER** description has already been given in previous topic for Sentiment Analysis using NLTK.

Import the necessary libraries as :

```
import tweepy
import pandas as pd
import matplotlib.pyplot as plt
import textblob
from nltk.sentiment.vader import SentimentIntensityAnalyzer
```

Now perform Twitter API authentication.

```
#Twitter API Authentication.
auth=tweepy.OAuth2BearerHandler("Your Bearer Token")
api=tweepy.API(auth)
```

Perform Sentiment Analysis on Twitter data.

```
# Sentiment Analysis
def sentiment_percentage(tweet_emotion_part,num_tweets):
    return 100*float(tweet_emotion_part)/float(num_tweets)

# Get the tweets on the basis of Hash Tags or Keywords.
search_tag=input("Enter the Hash Tag or Keyword for which you want to get the tweets :
")
no_of_tweets=int(input("How many tweets you want ? "))

# Iterate over the tweets.
tweets=tweepy.Cursor(api.search_tweets, q=search_tag).items(no_of_tweets)

positive_tweets=0
negative_tweets=0
neutral_tweets=0
polarity_of_tweets=0

# Lists to store positive, negative and neutral tweets.
tweets_list=[]
positive_tweets_list=[]
negative_tweets_list=[]
neutral_tweets_list=[]

for tweet in tweets:
    tweets_list.append(tweet.text)
    analysis=textblob.TextBlob(tweet.text)
    polarity_score=SentimentIntensityAnalyzer().polarity_scores(tweet.text)
    negative_score=polarity_score['neg']
    positive_score=polarity_score['pos']
    neutral_score=polarity_score['neu']
```

```
    compound_score=polarity_score['compound']

    polarity_of_tweets+=analysis.sentiment.polarity

    if negative_score>positive_score:
        negative_tweets_list.append(tweet.text)
        negative_tweets+=1
    elif positive_score>negative_score:
        positive_tweets_list.append(tweet.text)
        positive_tweets+=1
    elif positive_score==negative_score:
        neutral_tweets_list.append(tweet.text)
        neutral_tweets+=1

positive_tweets=sentiment_percentage(positive_tweets,no_of_tweets)
negative_tweets=sentiment_percentage(negative_tweets,no_of_tweets)
neutral_tweets=sentiment_percentage(neutral_tweets,no_of_tweets)
polarity_of_tweets=sentiment_percentage(polarity_of_tweets,no_of_tweets)

positive_tweets=format(positive_tweets,'.1f')
negative_tweets=format(negative_tweets,'.1f')
neutral_tweets=format(neutral_tweets,'.1f')
```

Plot the analyzed data.

```
# Printing Positive, Negative and Neutral Tweets.
print("Positive Tweets : ")
print(positive_tweets_list)

print("Negative Tweeets : ")
print(negative_tweets_list)

print("Neutral Tweets : ")
print(neutral_tweets_list)

# Plotting the data of Sentiment Alalysis.
labels=['Positive ['+str(positive_tweets)+'%]','Negative
['+str(negative_tweets)+'%]','Neutral ['+str(neutral_tweets)+'%]']
size=[positive_tweets,negative_tweets,neutral_tweets]
section_colors=['green','red','blue']
path,text=plt.pie(size,colors=section_colors,startangle=90)
plt.style.use('default')
plt.legend(labels)
plt.title("Sentiment Analysis on Twitter Data for "+search_tag)
plt.show()
```

Output :

```
Enter the Hash Tag or Keyword for which you want to get the tweets : amazonIN

How many tweets you want ? 10

Positive Tweets :
```

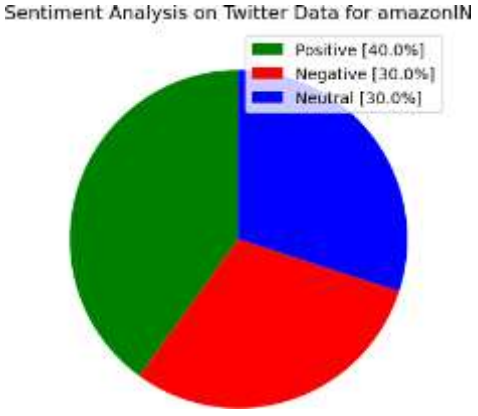
["Back to my weekend's favourite activity. Some insights\n\n56 days,\n12 emails\n>30 calls.\n\nAnd the amazing collaboratio… https://t.co/38JoPHM0aw", '@amazonIN i am unable to rest my Amazon password and i am trying to call 180030001593 on this number but every tim… https://t.co/QNEeqtA1cm', 'RT @amazonIN: Soundbar Days is back with exciting offers & great discount from popular brands! Get up to 55% off on bestselling soundbars,…', '@amazonIN @amazon @AmitAgarwal \nAmazon app is not performing well like add to cart, save later , move to cart obse… https://t.co/DPnt2Zg2pL']

Negative Tweeets :

["RT @AnandVe82668274: I'm unable to reapply and cancel my application. please solve this error ASPS. @amazonIN @AmazonHelp @ICICIBank @ICICI…", "I'm unable to reapply and cancel my application. please solve this error ASPS. @amazonIN @AmazonHelp @ICICIBank… https://t.co/4P6fjkTuTE", 'RT @IqooInd: Switching back to black for the extraordinary and unmatched tempting looks.\nAnd that’s just the starting point when it comes t…']

Neutral Tweets :

['@indusos @amazonIN □□□□\n App is More Than an App for Me.\n□□□□□□□□□□□□\n\n#Giveaway\n#AppSeBhiZyada #Giveaways #Contest… https://t.co/LkIcqke37I', '@IqooInd -> Qualcomm® Snapdragon™ 888+ 5G Mobile Platform. \n#iQOORaidNights \n@IqooInd @iqooesports \n@amazonIN', '@motorolaindia The #MotorolaEdge30Pro have the   indias beast&fastest snapdragon 8 Gen1 processor… https://t.co/yrhrQNLUFY']



Downloading the twitter datasets online : The online available datasets containing Twitter data can be downloaded and different analytics can be performed on it.

Example : https://www.kaggle.com/crowdflower/twitter-user-gender-classification

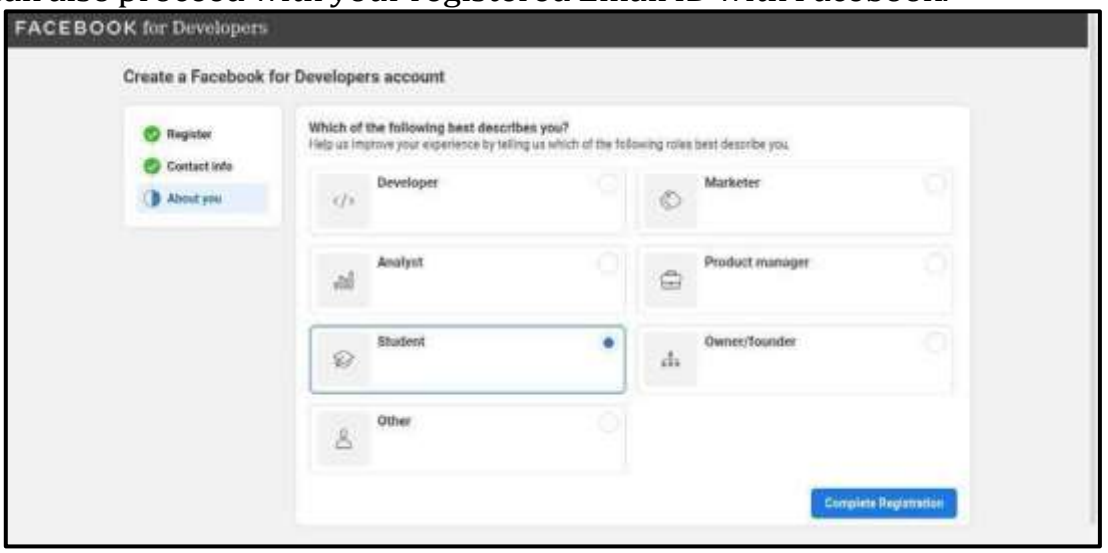
Self-Activity : Download and analyze the data using above link and apply different analytics techniques on it.

Facebook Data Analysis :

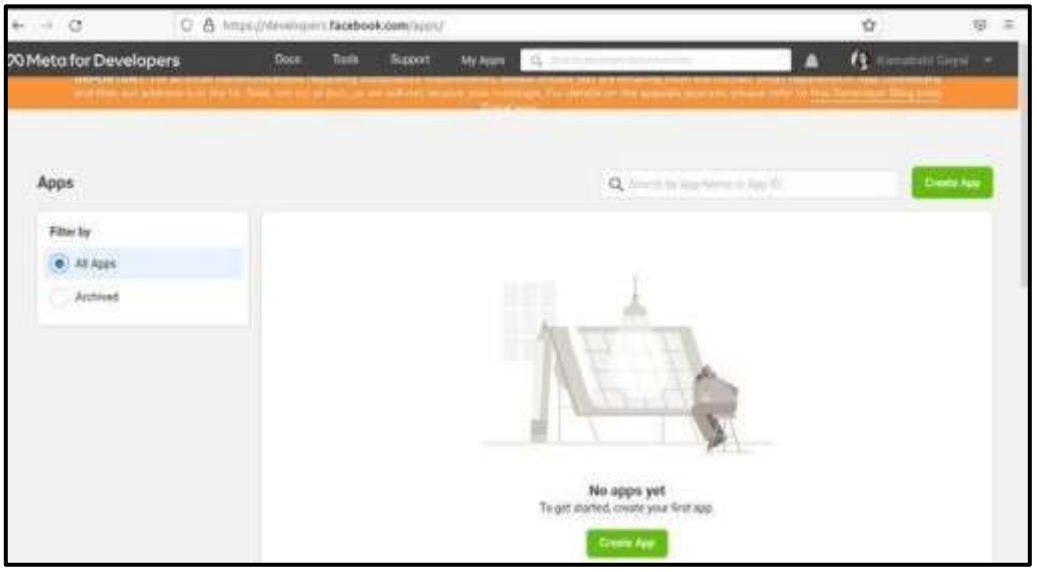
✓ To get Facebook data to perform analysis, there are multiple ways. Some of these are :
- Getting data using Facebook Access Token.
- Downloading data directly from a Facebook Account.
- Getting Facebook data from online available datasets on Kaggle.

Getting data using Facebook Access Token

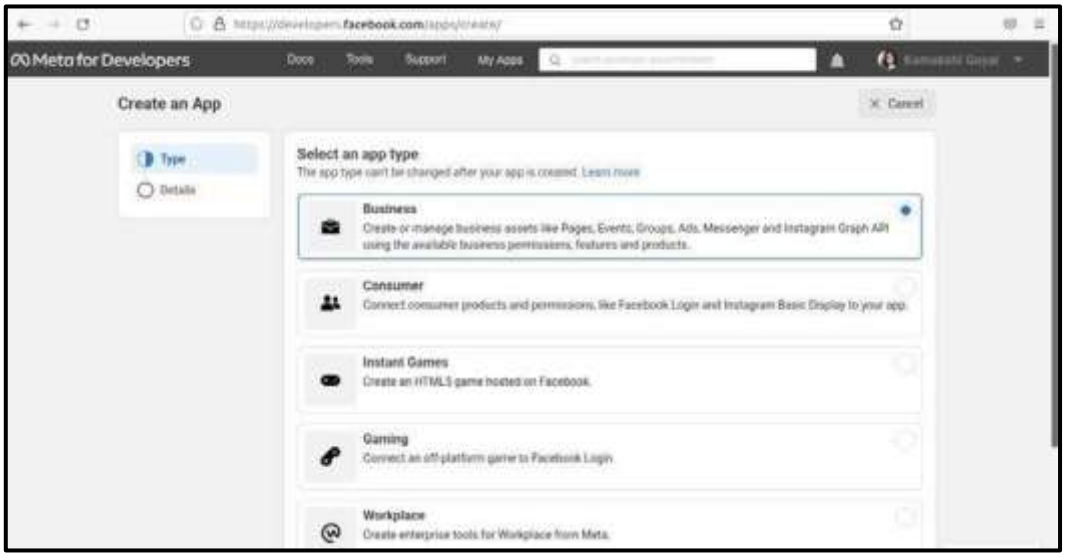
✓ For this, Facebook's developer account is needed.
✓ Visit the link : https://developers.facebook.com/
✓ Click on **Get Started**.
✓ Then provide the Email ID you want to associate with your Facebook Developer account and click on **Send Verification Email**.
✓ Or you can also proceed with your registered Email ID with Facebook.

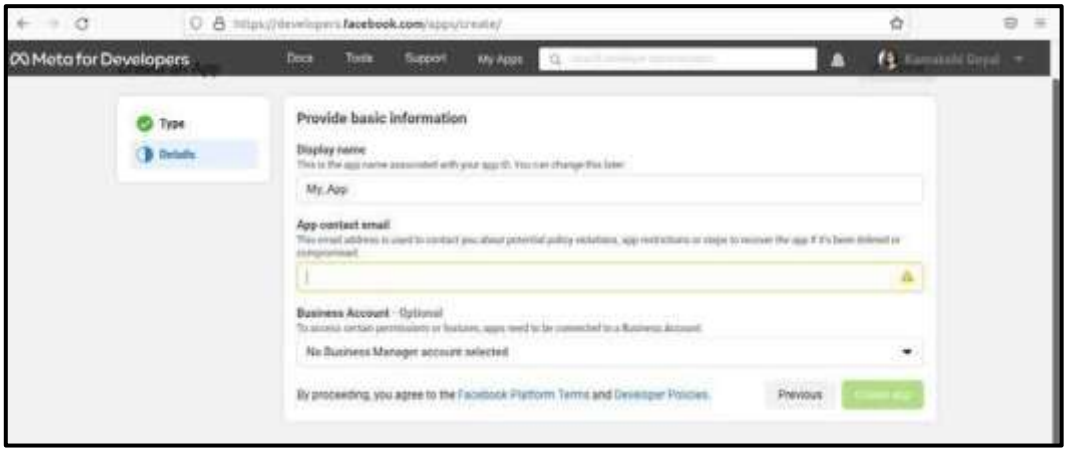


✓ Now create an app to get the token to be used for further processing.



✓ Click on **Create App** and then select the type of app to be created. Multiple options will be available i.e. Business, Consumer, Instant Games, Gaming, Workplace, None. You can read the details and select an option. If **Business** option available in list is selected, then it creates an app which manages business assets like Pages, Events, Groups, Ads, Messenger and Instagram Graph API using the available business permissions, features and products.



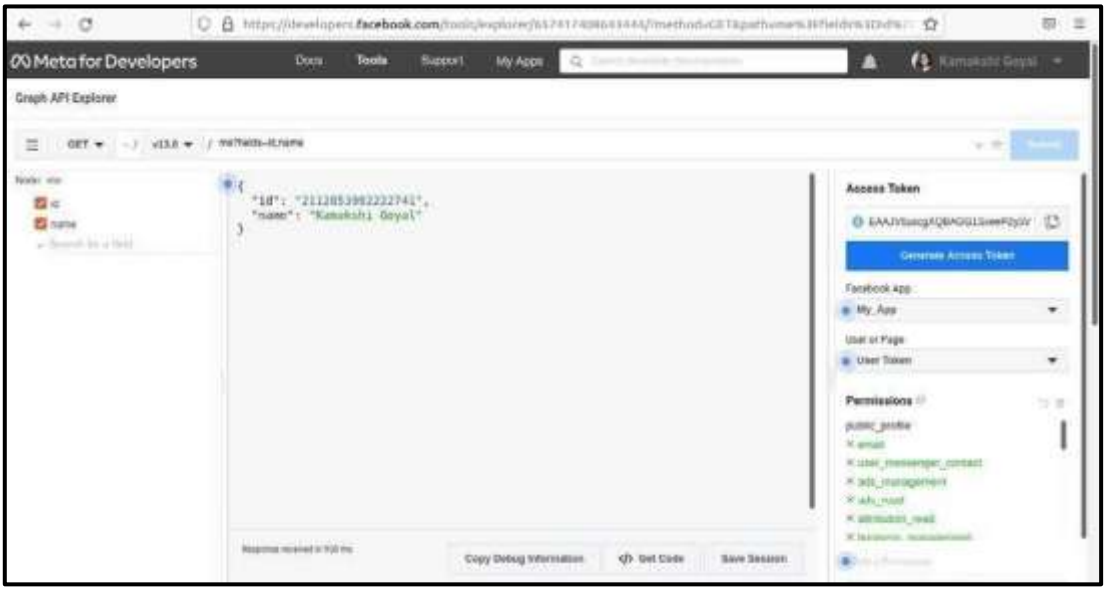


✓ After an app gets created, you can get the Access Token as follows :

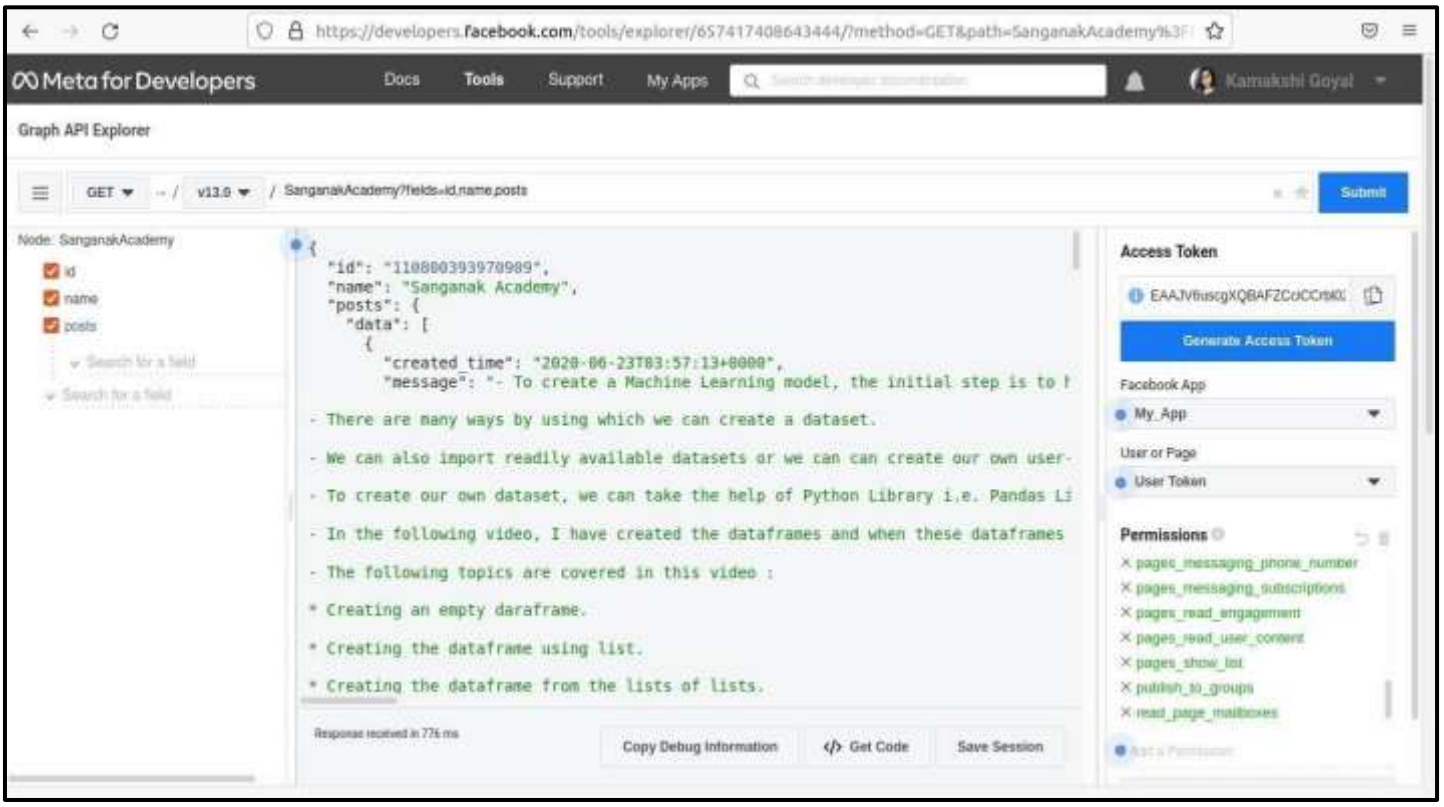
- Go to : https://developers.facebook.com/tools/explorer/

- Now click on **Generate Access Token**. Proceed to the next step by clicking on **Continue**.
- The **Access Token** will be visible in Access Token input box.

- Now allow the necessary permissions to access the Facebook pages as well.
- Click on "Add a Permission" dropdown and select the permissions from it.



- From "User or Pages" dropdown select "Get User Token" again to get the Token with revised permissions.
- Now if we want to see the details of publically available Facebook users or pages, change the request in the request url box.
  Example : If we want to get the details of Facebook Page "Sanganak Academy" then change the name of page as : **SanganakAcademy?fields=id,name**. Before that, make sure to allow the permissions for accessing that page as well.
- You can see the posts on this page as well by changing the url as : **SanganakAcademy?fields=id,name,posts**. Same you can do to access other information as well.



✓ The Facebook data can be accessed using Python script as follows :

❖ Import the necessary libraries :

```
import requests
import time
import pickle
import random
```

❖ Provide Access Token and get the URL to access the data :

```
# Access Token
access_token="Your Access Token"
# In Graph URL, provide the correct version of Graph API. Here currently I am using
v13.0
graphURL="https://graph.facebook.com/v13.0/"
# Request URL to get the relevant data of Facebook Page Sanganak Academy.
# You can access any other page by using its ID as well.
requestURL="SanganakAcademy?fields=id,name,posts{message,created_time,comments.limit(
0).summary(true), likes.limit(0).summary(true)}"
actual_url=graphURL+requestURL
```

❖ Call the Graph API using get method of requests library.

```
result=requests.get(actualURL,{'access_token':access_token})
```

❖ Getting posts from a Facebook page.

```
# To get all posts.
result=result.json()['posts']
print(result['data'])
# To get individual post
print(result['data'][0])
```

❖ Create a DataFrame using Pandas library.

```
# Create a dataframe using pandas library
import pandas as pd
fb_dataframe=pd.DataFrame(received_data)
fb_dataframe=pd.json_normalize(received_data)
print(fb_dataframe)
# Columns
print("Columns in Dataframe : ")
for col in fb_dataframe.columns:
    print(col)
```

Here columns in the dataframe are :

- message
- created_time
- id
- comments.data
- comments.summary.order
- comments.summary.total_count
- comments.summary.can_comment
- likes.data
- likes.summary.total_count
- likes.summary.can_like
- likes.summary.has_liked

❖ To find Top Liked Post.

```
# To find top liked post.
top_liked_post=fb_dataframe[fb_dataframe['likes.summary.total_count']==fb_dataframe['
likes.summary.total_count'].max()]
print("Top Liked Post : ")
print(top_liked_post['message'])
```

Similarly we can find the top commented post as well.

❖ Grouping Facebook posts by date on which they are created.

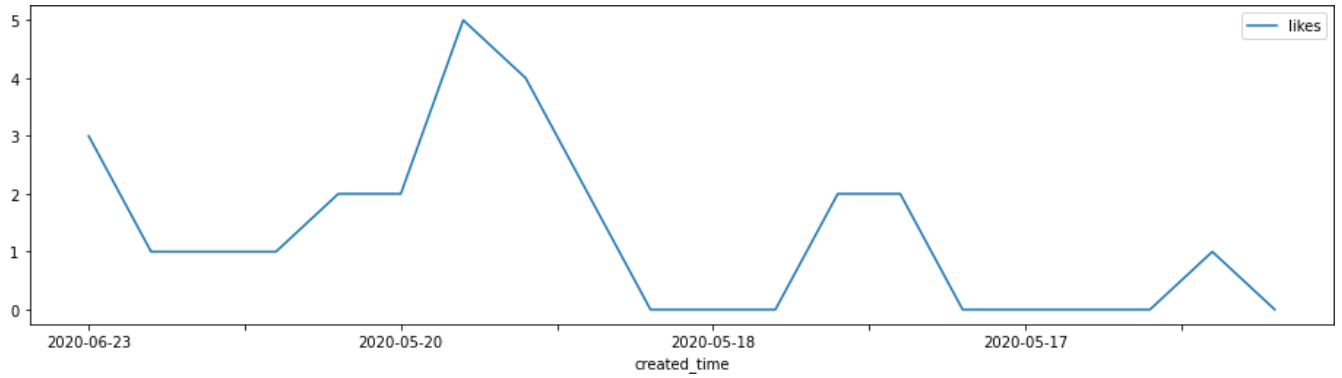
For this, use the dataframe column “created_time”. It contains datetime format data. So date can be obtained by splitting it in two sections i.e. date and time.

```
# Grouping facebook posts by date.
fbdata_with_dates=pd.DataFrame(fb_dataframe[['created_time','message','likes.summary.
total_count','comments.summary.total_count']])
# Split date and time.
date=fbdata_with_dates['created_time'].str.split('T')
fbdata_with_dates['created_time']=date.str[0]
print(fbdata_with_dates)
grouped_data=fbdata_with_dates.groupby('created_time').sum()
```

Visualize the data :

```
# Visualizing the data.
import matplotlib.pyplot as plt
# Time Series
```

```
time_likes=pd.Series(data=fbdata_with_dates['likes.summary.total_count'].values,index
=fbdata_with_dates['created_time'])
time_likes.plot(figsize=(16,4),label="likes",legend=True)
plt.show()
```

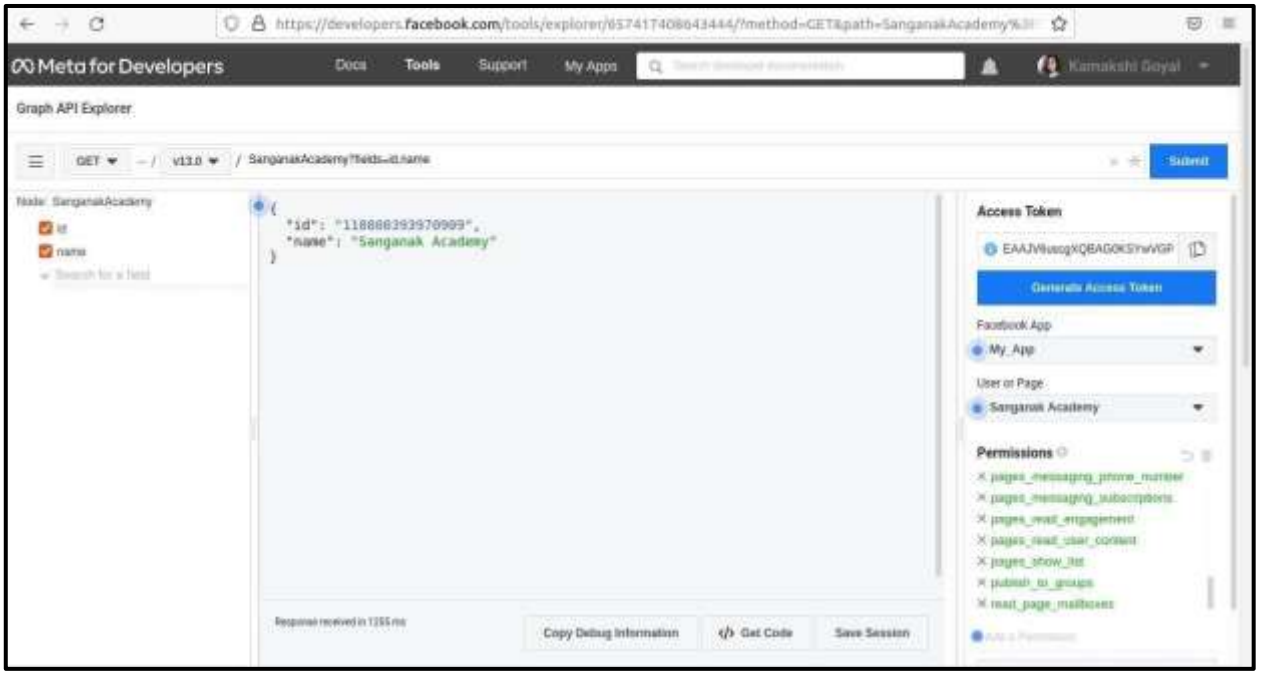


Automating Facebook Programmatically

(**Note :** To analyze the Facebook data, we can use User Access Token, but to add, update or delete something on live Facebook Page, we need Page Access Token. To get page access token, make sure there is a Facebook Page associated with your Facebook Account.)

Steps to get Page Access Token :

✓ To get page access token, select your Facebook Page name on "User or Page" option.



✓ And then click on "Generate Access Token". Now copy the generated access token and use it for further analysis.

❖ **Creating a Facebook post and Commenting on it :** Consider the following steps to create a post and comment on a specific Facebook Page on your own account :

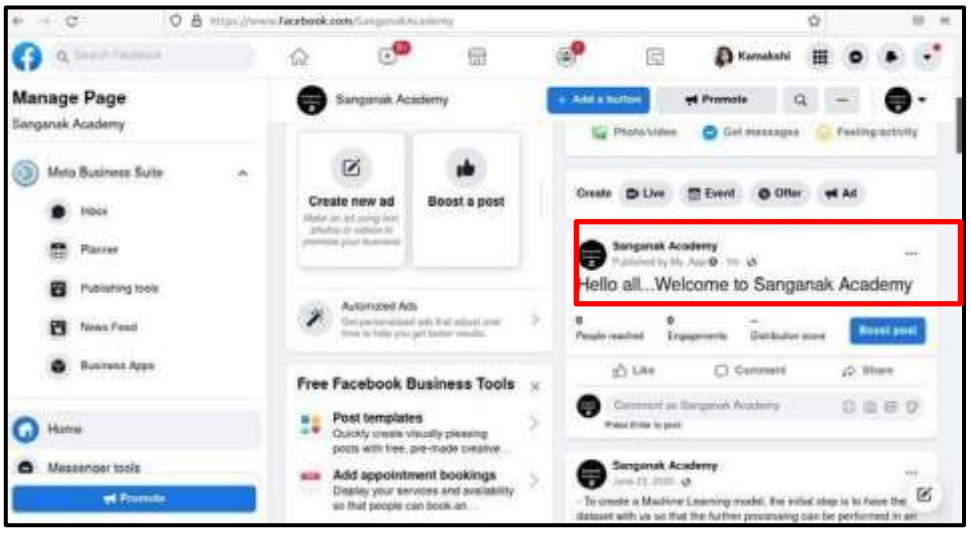
✓ For this, we need **facebook-sdk** library. To install facebook library, use :

```
pip install facebook-sdk
```

✓ **Creating a Facebook post :** To post something on Facebook Page wall, use put_object() method of Facebook Graph API.

```
import facebook
access_token="Your Page Access Token"
```

```
fb=facebook.GraphAPI(access_token)
fb.put_object(parent_object='me', connection_name='feed', message='Hello all...Welcome to Sanganak Academy')
```



✓ **Commenting on a Facebook post :** To comment on a specific post, "Post ID" and "Facebook Page ID" is needed. To get the Facebook Post ID, go to the post and click on the date on which post is created. Inside the website URL, you will see the post ID at last.
Example : https://www.facebook.com/SanganakAcademy/posts/511268930590718

Here, 511268930590718 is the Facebook Post ID. To get "Facebook Page ID", Open the Facebook Page and in **About** section, you will find the Facebook Page ID.

Now combine Facebook Page ID and Facebook Post ID as pageid_postid. For example : If Page ID = 12345 and Post ID = 511268930590718, then combine it as 12345_511268930590718.

Syntax : `graph_api_object.put_object(parent_object = 'post_id', connection_name= 'comments', message = 'Your comment')`

Consider the following script to comment on a post.

Example :

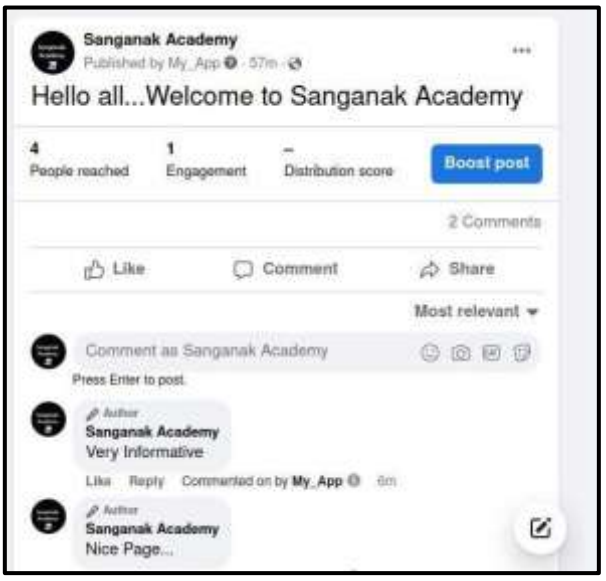
```
# Writing a comment on a facebook post.
fb.put_object(parent_object='12345_511268930590718',connection_name='comments',message='Nice Page...')
```

You can also comment on a specific post using put_comment() method as :

Syntax : `graph_api_object.put_comment(object_id = 'post_id', message = 'Your comment')`

Example :

```
# Writing a comment on a facebook post.
fb.put_comment(object_id='12345_511268930590718', message='Very Informative')
```



❖ Liking a post :

Syntax : `graph_api_object.put_like(object_id = 'post_id')`

Example :

```
# Liking a facebook post.
fb.put_like(object_id='12345_511268930590718')
```

❖ Deleting a Facebook post :

Syntax : `graph_api_object.delete_object(id = 'post_id')`

Example :

```
# Deleting a facebook post.
fb.delete_object(id='12345_511268930590718')
```

The Facebook post will be deleted.

❖ Getting all friends of an active user :

```
# Get the active user's friends.
friends = fb.get_connections(id='me', connection_name='friends')
```

(Refer this https://facebook-sdk.readthedocs.io/en/latest/api.html for more information about Facebook SDK.)

YouTube Data Analysis :

✓ YouTube data can be gathered from multiple sources such as :
- Using YouTube API.
- Downloading already available datasets on Kaggle.

Consider the dataset : https://www.kaggle.com/datasnaek/youtube-new?select=CAvideos.csv

❖ Read the dataset

```
import pandas as pd
youtube_data=pd.read_csv('CAvideos.csv')

# Get the columns in a dataset
print(youtube_data.columns)
```

❖ To find total views, likes, dislikes and comment count

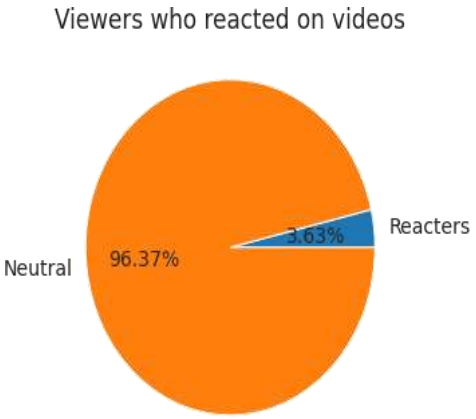
```
# To find total views, likes, dislikes and comment count.
print(youtube_data[['views','likes','dislikes','comment_count']].sum())
```

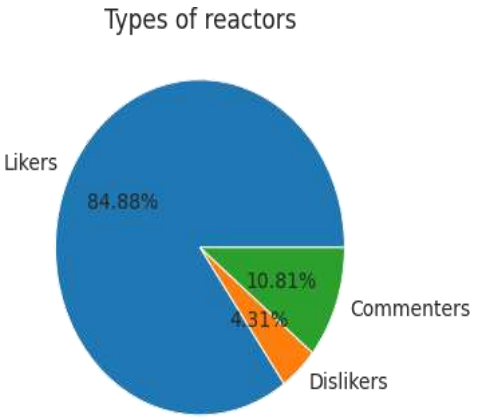
❖ Perform statistical analysis on YouTube data

It will plot the data for :
- Finding the viewers who reacted on videos.
- To classify the reactors on videos as “Likers”, “Dislikers” and “Commenters”.

```
# performing statistical analysis on youtube data.
import seaborn as sns
import matplotlib
import matplotlib.pyplot as plt

sns.set_style('darkgrid')
matplotlib.rcParams['font.size'] = 14
matplotlib.rcParams['figure.figsize'] = (12, 5)
matplotlib.rcParams['figure.facecolor'] = '#00000000'

fig = plt.figure()

axis1 = fig.add_axes([0, 0, 0.75, 0.75], aspect=1) # add_axes([left, bottom, width,
height],aspect=1)

# To find viewers who reacted on videos.
pie_vars = ['Reacters','Neutral'];
pie_values =
[youtube_data['likes'].sum()+youtube_data['dislikes'].sum(),youtube_data['views'].sum()-
(youtube_data['likes'].sum()+youtube_data['dislikes'].sum())]
axis1.pie(pie_values,labels=pie_vars,autopct='%1.2f%%')
axis1.set_title("Viewers who reacted on videos")
```

```
axis2 = fig.add_axes([0.8, 0, 0.75, 0.75], aspect=1)
# Pie chart for reactors
pie_vars = ['Likers','Dislikers','Commenters']
pie_values =
[youtube_data['likes'].sum(),youtube_data['dislikes'].sum(),youtube_data['comment_count'
].sum()]
axis2.pie(pie_values,labels=pie_vars,autopct='%1.2f%%')
axis2.set_title("Types of reactors")

plt.show()
```





❖ Find top most viewed videos

```
# To find top 5 most viewed videos.
print(youtube_data.sort_values(by='views',ascending=False).head(5))
```

❖ Find least viewed videos

```
# To find top 5 least viewed videos.
print(youtube_data.sort_values(by='views',ascending=True).head(5))
```

❖ Find top most liked videos

```
# To find top 5 most liked videos.
print(youtube_data.sort_values(by='likes',ascending=False).head(5))
```

❖ Find least liked videos

```
# To find top 5 least liked videos.
print(youtube_data.sort_values(by='likes',ascending=True).head(5))
```

❖ Performing Year wise Statistics

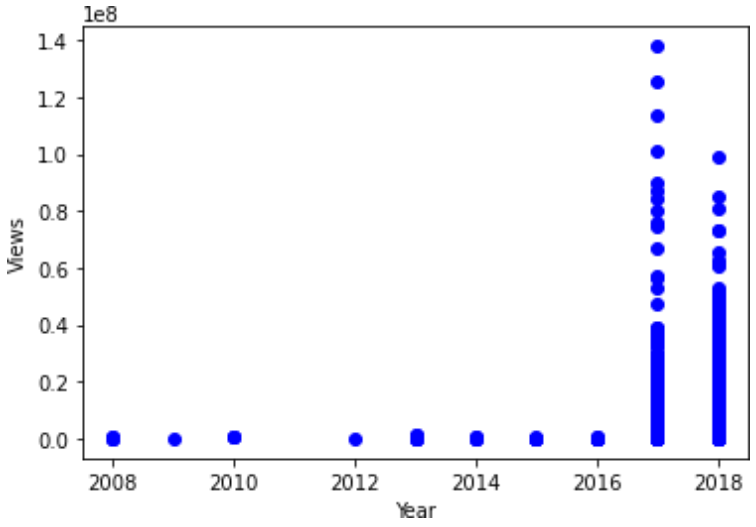
First clean the column where we need to work with date. In above dataset, “publish_time” column contains data in datetime format. To perform statistics year wise or month wise, split the date in separate sections as year, month and day.

```
# Split the date into year, month and date.
import datetime
i=0
for i in range(youtube_data.shape[0]):
```

```
    date_time_obj = datetime.datetime.strptime(youtube_data['publish_time'].at[i],'%Y-%m-%dT%H:%M:%S.000Z')
    youtube_data['publish_time'].at[i] = date_time_obj
    i = i+1

date=[]
year=[]
month=[]
day=[]

for i in range(youtube_data.shape[0]):
    d = youtube_data['publish_time'][i].date()
    y = youtube_data['publish_time'][i].date().year
    m = youtube_data['publish_time'][i].date().month
    days = youtube_data['publish_time'][i].date().day
    date.append(d) # Storing dates
    year.append(y) # Storing years
    month.append(m) # Storing months
    day.append(d) # Storing days
    i = i+1
youtube_data.drop(['publish_time'], inplace=True,axis=1)
youtube_data['publish_time']=date
youtube_data['year']=year
youtube_data['month'] = month
youtube_data['day'] = day
```

Year wise statistics of views :

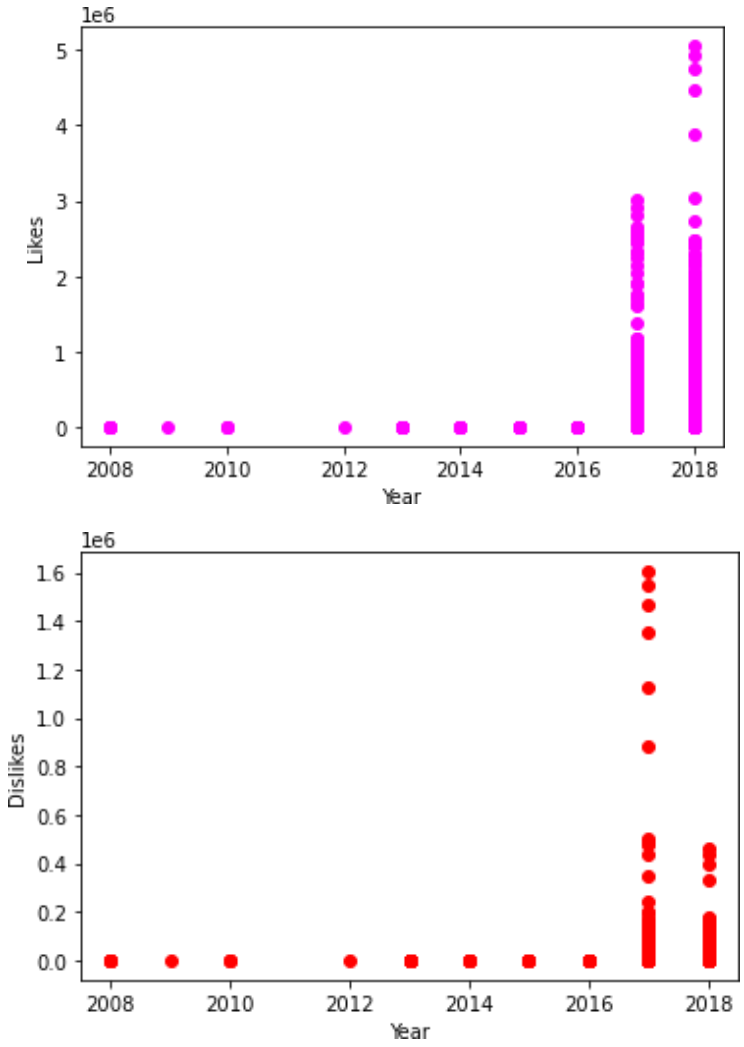
```
# Year wise statistics of views.
plt.scatter(youtube_data['year'], youtube_data['views'], c="red")
plt.xlabel("Year")
plt.ylabel("Views")
plt.show()
```



Year wise statistics of likes and dislikes

```
# Year wise statistics of likes.
plt.scatter(youtube_data['year'], youtube_data['likes'], c="magenta")
plt.xlabel("Year")
plt.ylabel("Likes")
plt.show()
```

```
# Year wise statistics of dislikes.
plt.scatter(youtube_data['year'],
youtube_data['dislikes'], c="red") plt.xlabel("Year")
plt.ylabel("
Dislikes")
plt.show()
```



Lab Assignments

SET A

1. Consider any text paragraph. Preprocess the text to remove any special characters and digits. Generate the summary using extractive summarization process.
2. Consider any text paragraph. Remove the stopwords. Tokenize the paragraph to extract words and sentences. Calculate the word frequency distribution and plot the frequencies. Plot the wordcloud of the text.
3. Consider the following review messages. Perform sentiment analysis on the messages.
   i. I purchased headphones online. I am very happy with the product.
   ii. I saw the movie yesterday. The animation was really good but the script was ok.
   iii. I enjoy listening to music
   iv. I take a walk in the park everyday

4. Perform text analytics on WhatsApp data :

Write a Python script for the following :

i. First Export the WhatsApp chat of any group. Read the exported ".txt" file using open() and read() functions.

ii. Tokenize the read data into sentences and print it.

iii. Remove the stopwords from data and perform lemmatization.

iv. Plot the wordcloud for the given data.

Set B

1. Consider the following dataset :
https://www.kaggle.com/datasets/prasertk/top-1000-instagram-influencers Write a Python script for the following

i. Read the dataset and find the top 5 Instagram influencers from India.

ii. Find the Instagram account having least number of followers.

iii. Read the column "Category", remove stopwords and plot the wordcloud to find the keywords which will imply that in which category maximum accounts are created.

iv. Group the Instagram accounts category wise.

v. Visualize the dataset and plot the relationship between Followers and Authentic engagement columns.

2. Consider the following dataset :
https://www.kaggle.com/datasets/seungguini/youtube-comments-for-covid19-related- videos?select=covid_2021_1.csv
Write a Python script for the following :

i. Read the dataset and perform data cleaning operations on it.

ii. Tokenize the comments in words.

iii. Perform sentiment analysis and find the percentage of positive, negative and neutral comments.

3 Consider the following dataset :
https://www.kaggle.com/datasets/datasnaek/youtube-new?select=INvideos.csv Write a Python script for the following :

i. Read the dataset and perform data cleaning operations on it.

ii. Find the total views, total likes, total dislikes and comment count.

iii. Find the least and topmost liked and commented videos.

iv. Perform year wise statistics for views and plot the analyzed data.

v. Plot the viewers who reacted on videos.

Set C

Q.2 Write a Python script to read the Tweets using Twitter API and tweepy library to perform the following tasks :

i. Authenticate Twitter API (Using Bearer Token)
ii. Get the tweets using Keywords or Hash Tags.
iii. Find the total number of likes and retweets on each tweet.
iv. Find the most liked tweet and print its text.
v. Visualize the tweets and plot the time series for likes and retweets along with dates on which tweets are published.

Download the data directly from a Facebook Account

- First log in to your Facebook account.
- Then go to **Settings** -> **Your Facebook information** -> **Download your information**
- Then click on View link to download the data in JSON format. Select what information you want to download.
- You can select multiple options available to download.
- For example, if we want to work with Posts on Facebook, then select posts option and then click on
  Request a download button. You can select any number of information and download it.
- The data will be available in separate folders in JSON format.

1. Import and format the data into a DataFrame using pandas library. Example : For working with Facebook Posts, read the JSON file available in "Posts" folder as :

```
import pandas as pd
facebook_dataframe=pd.read_json("your_posts.json")
```

Similarly you can work with other downloaded data and read the JSON files available in them.

2. Now perform data cleaning operation on created dataframe and remove unnecessary columns.
3. Perform multiple statistical analysis such as finding the posts by date, number of likes on a post, comments on a post.
4. Perform sentiment analysis to find the polarity scores and classify the posts text in three categories i.e. positive, negative and neutral posts.

Signature of the instructor _____ Date _____

Assignment Evaluation

0: Not done _____ 2: Late Complete _____ 4: Complete _____

1: Incomplete _____ 3: Needs improvement _____ 5: Well Done _____